VPN-Encrypted Network Traffic Classification Using a Time-Series Approach

Jaidip Kotak, Idan Yankelev, Idan Bibi, Yuval Elovici, and Asaf Shabtai

***Abstract*—Network traffic classification provides value to organizations and Internet service providers (ISPs). The identification of applications or services from network traffic enables organizations to better manage their business, and ISPs to offer services to their users. Given the vast quantity of traffic flowing in and out of organizations, it is impractical to write manual signatures for traffic identification. The effectiveness of machine learning (ML) in the identification of applications or services from network traffic has been demonstrated. Even when network traffic is encrypted, ML algorithms achieve high accuracy in the task of traffic identification based on statistical information and the packets' headers and payloads. However, existing approaches were shown to be ineffective for VPN-encrypted network traffic. In this study, we propose a novel time-series based approach for the identification of traffic/source applications on VPN-encrypted traffic. We also demonstrate the broad applicability of our proposed approach by evaluating its effectiveness on non-VPN traffic that is encrypted, and on IoT traffic.**

***Index Terms*—Network traffic classification, virtual private networks (VPN), machine learning, encrypted traffic and cybersecurity.**

I. Introduction

AMONG other changes in the past decade, today's work-from-home culture has played a role in shifting people to online platforms, which has led to a dramatic increase in the amount of Internet traffic. The number of Internet users worldwide is currently around 4.9 billion; this means that almost two-thirds of the world's population is connected to the Internet [1].

Due to online privacy campaigns and growing concern from advertisers, Internet service providers (ISPs), and governments regarding the disruption, manipulation, and monitoring of Internet traffic, users' awareness of privacy issues associated with Internet use has grown [2], [3], [4], [5], [6], [7]. To preserve privacy, circumvent censorship, and access geo-filtered content, many users use virtual private network (VPN) technology [8], [9], [10].

Given the increased volume of network traffic, there is a need for service providers and organizations to perform automatic network traffic classification (NTC) to infer the applications or the type of applications, by analyzing the packets. This capability is extremely important for organizations and network service providers, since it (1) provides effective security (e.g., detecting anomalous traffic or application behavior), and (2) guarantees network quality-of-service (QoS) for specific applications (e.g., video streaming).

The main challenges faced when performing NTC are: (1) the large volume of network traffic, (2) the evolving nature of network traffic patterns due to user behavior and changes in applications' functionality, and (3) the use of technologies like network address translation (NAT), internal proxies, and VPNs.

The two existing network traffic classification approaches—feature-based and rule-based methods (statistical and behavioral)—are not directly applicable to VPN-encrypted traffic. The limitation of the feature-based approach is based on the network flow, from which it identifies the right features and preprocesses them for the machine learning (ML) model [11], [12]; it is not possible to segregate different network flows in VPN-encrypted traffic, as all of the traffic is within a single VPN-encrypted tunnel and is therefore a single network flow. The rule-based approach relies mainly on the port number and the header fields of network communication [13], [14]. However, as mentioned, in VPN-encrypted traffic there is a single network flow and the header information is not visible due to the encryption provided by the VPN. Therefore, these approaches are not directly applicable to VPN-encrypted network traffic classification (V-NTC).

Our Method: We propose a method for V-NTC, which is based on the sliding window of a time series. In this method, features from the packets are derived every second, and the features of m seconds are appended to form a single training or testing instance, where m is the window length. Then, to form the next data instance, we slide by one second (i.e., 2 to $m+1$ seconds) and construct a second data instance. The InceptionTime model [15] was used to classify the time-series based instances.

Evaluation: Initially, we planned to use the publicly available VPN traffic dataset – ISCXVPN2016 [16] – to evaluate the proposed method for VPN-encrypted traffic classification. However, since we found a discrepancy in this dataset (described in detail in Section II-B), we formed our own dataset that includes traffic from three geographical locations. This dataset is available at https://zenodo.org/records/13301379. We evaluated our approach on the entire dataset and for each geographical location, on both the classification task at the category level (e.g.,

Received 25 December 2023; revised 12 August 2024 and 22 November 2024; accepted 23 January 2025. Date of publication 20 February 2025; date of current version 22 April 2025. The associate editor coordinating the review of this article and approving it for publication was Y. Diao. *(Corresponding author: Jaidip Kotak.)*

The authors are with the Department of Software and Information Systems Engineering, Ben-Gurion University of the Negev, Be'er Sheva 84105, Israel (e-mail: jaidip@post.bgu.ac.il).

Digital Object Identifier 10.1109/TNSM.2025.3543903

streaming, chat), and the application level (e.g., YouTube, Vimeo). An accuracy of 93-95% was obtained for category-level classification for each location and for the combined traffic from all three locations. Accuracy ranging from 84% to 91% was obtained for application-level classification for each location and when combining the traffic from all three locations. To demonstrate our method's generic nature on non-VPN traffic, we evaluated it on the ISCXVPN2016 dataset and a subset of the IoTTrace [17] dataset.

Contribution: The contributions of our research are as follows:

- To the best of our knowledge, we are the first to classify and identify VPN-encrypted traffic using a time-series approach.
- The proposed approach is generic in nature, i.e., it is applicable to VPN-encrypted traffic, non-VPN traffic, and IoT device type classification, as demonstrated in our evaluation on different datasets.
- The proposed approach can be used on network traffic originating from different geographical locations (using a VPN).
- We created a dataset of VPN-encrypted and non-VPN traffic from three different geographical locations which contains more unique applications than existing datasets and will make this dataset available to the research community.

The remainder of this paper is organized as follows. Section II reviews the related work on VPN traffic classification and the existing public VPN dataset. Section III describes the characteristics of the VPN-encrypted traffic dataset, the network setup during data generation, and the data generation process. Section IV outlines the methods used, including data preprocessing, feature construction, and the classification model. Section V presents the experiments conducted and their results. Section VI discusses the findings, limitations, and potential areas for future research. Finally, Section VII concludes the paper by summarizing our findings and contributions.

II. Related Work

A. Network Traffic Classification

In the past, network traffic detection was performed based on the port numbers used in network communication. The same port numbers were used by all the applications that were communicating on the same protocol [18]. Therefore, it was easy to identify the network traffic, as mentioned in [19], [20], [21]. Later, payload-based deep packet inspection (DPI) tools such as PACE, OpenDPI, L7-filter, nDPI and Cisco's NBAR were used to extract features from packets' payloads; and then, simple ML models (such as decision tree (DT), random forest (RF), and k-nearest neighbors (KNN)) capable of bifurcating between the different traffic groups and classifying the packets based on the features were used, as mentioned in [22], [23], [24], [25], [26], [27].

However, as the Internet evolved, the above-mentioned methods have become obsolete, since most developers today do not follow a specific standard in selecting the port number for their application's communication. Even DPI tools are irrelevant, as most of the traffic is encrypted or VPN-encrypted these days.

Due to the lack of privacy, technologies such as NAT, internal proxies, and VPNs were established to allow companies, organizations, and individuals to use a network without revealing any identifying data; as a result, new methods capable of performing encrypted NTC with high accuracy without compromising users' privacy are needed. Advances in the field led to the development of novel approaches based on network traffic flow features, well suited to the data's encrypted nature, as well as the use of deep neural network methods which allow the computer to find its own weights in order to correctly classify the traffic classes.

In [28], the authors proposed a framework that employs a stacked autoencoder (SAE) and convolution neural network (CNN) to classify network traffic. To train the neural networks, they performed a preprocessing phase in which the Ethernet header and uniformed protocol header length were removed, and the packets' bits were transformed to bytes; afterward, they filtered out all non-application data-related packets, such as packets with ACK/SYN/FIN flags or DNS queries. The framework achieved 98% accuracy in the application identification task and 93% in the traffic categorization task by using the CNN network as the classification model. Similar work has been performed by [30], [31], [34], [39], [41] and [40] who utilized payload and other header information, applying deep learning models like a CNN, 1D-CNN & bidirectional gated recurrent unit (BiGRU), bidirectional encoder representation Transformer (BERT) & CNN, three-layer multilayer perception (MLP), CNN, and ensemble long short-term memory (LSTM), respectively, and achieving results in a similar range. However, the major limitation of these approaches is that they rely on the presence of clear text information in encrypted traffic, particularly with the TLS protocol, where initial negotiation happens in clear text. In real VPN traffic, however, this information is encrypted and hidden within the VPN tunnel, making these methods bound to fail.

In addition to deep learning models, several studies have leveraged machine learning algorithms for classifying VPN and non-VPN network traffic. For instance, [32] proposed a method that utilized time-related statistical features generated by ISCXFlowMeter/CICFlowMeter, such as flow duration, flow bytes per second, max active time, max biat (backward inter-arrival time), max flowiat (flow inter-arrival time), mean biat, min active time, min biat, and std active time (standard deviation of active time). These features were then used to train a RF model, achieving over 95% accuracy for category-level VPN traffic classification.

ISCXFlowMeter/CICFlowMeter is a network traffic flow generator and analyzer that processes packet capture (PCAP) files to generate bidirectional flows. Each flow is identified based on the direction of the first packet (source to destination or destination to source) and enriched with over 80 statistical features, including flow duration, packet counts, byte counts, and packet lengths for both forward and backward directions. These features are frequently used in network traffic classification tasks, enabling the detection and differentiation of

TABLE I
SUMMARY OF ENCRYPTED TRAFFIC CLASSIFICATION APPROACHES

Ref	Year	Classifier	Features	Dataset used	Assumes VPN traffic can be split into flows	Uses application layer payload information
[28]	2017	SAE & CNN	Application layer payload of packets	ISCXVPN2016	✓	✓
[29]	2018	LSTM	Header fields & application layer payload of packets	ISCXVPN2016	×	✓
[30]	2020	CNN	Application layer payload of packets	ISCXVPN2016	✓	✓
[31]	2021	1D-CNN & BiGRU	Header fields & application layer payload of packets	ISCXVPN2016	✓	✓
[32]	2022	RF	CICflowmeter features	ISCXVPN2016	✓	×
[33]	2022	LSTM & ODENet	Packet sizes and inter-arrival times	ISCXVPN2016 & Private	✓	×
[34]	2023	BERT & CNN	Application layer payload of packets	ISCXVPN2016	✓	✓
[35]	2023	NN & SVM	CICflowmeter features	ISCXVPN2016 & Private	✓	×
[36]	2023	AAE & DSVDD	Packet sizes and inter-arrival times	ISCXVPN2016	✓	×
[37]	2023	Various boosting models (e.g., XGBOOST, LGBM, AdaBoost)	CICflowmeter features	ISCXVPN2016	✓	×
[38]	2023	CNN	Statistical features and markov chain-based probability features.	ISCXVPN2016	✓	✓
[39]	2023	Three-layer MLP	Application layer payload of packets	ISCXVPN2016 & Tor & USTC-TFC	×	✓
[40]	2024	LSTM & ensemble	Header fields & application layer payload of packets	Private	✓	✓
Ours	2024	InceptionTime	Statistical time-series feature based on relative time and packet length of incoming and outgoing packets.	Our dataset & ISCXVPN2016 & IoTTrace	×	×

various traffic types. However, it is important to note that these features rely on the ability to aggregate the packets by flows. This dependency makes such methods unsuitable with datasets where multiple flows are encapsulated within a VPN tunnel, appearing as a single flow.

Similarly, studies like [35] and [37] have employed features derived from ISCXFlowMeter with machine learning models such as neural networks (NN), support vector machines (SVM), and boosting algorithms (e.g., XGBoost, LightGBM, AdaBoost), achieving comparable results.

The limitation of such studies is that ISCXFlowMeter/CICFlowmeter assume that traffic can be segmented into flow records using 5-tuple values (source IP, destination IP, source port, destination port, and protocol). However, this assumption does not hold true for real VPN traffic, where traffic flows are often merged and encrypted in the VPN tunnel. As a result, these methods are not directly applicable to real VPN traffic scenarios.

Additionally, studies such as [33] and [36] utilized only the packet sizes and inter-arrival times for classification, employing LSTM & ordinary differential equation networks (ODENet) and adversarial auto-encoder (AAE) & deep support vector data description (DSVDD), respectively. However, these approaches assume that traffic can be neatly segmented into flows, which limits their applicability to real-world VPN scenarios where this assumption does not hold true.

In contrast, Vu et al., [29] developed a time-series feature extraction method consisting of a feature engineering method, which is used to extract significant attributes of the encrypted network traffic behavior by analyzing the time series of receiving packets, and a deep learning-based method which is used to exploit the correlation of time-series data samples of the encrypted network applications. The authors used the newly generated time-series features to train a model based on the LSTM recurrent neural network to learn the features in the time-series samples. The method's performance surpassed that of methods proposed in prior studies, obtaining an accuracy of 98.17% while reducing the number of features used to 55; in contrast, 1,500 features were required in the model proposed in [28]. The method was primarily tested on category classification instead of application classification.

Table I summarizes recent work on encrypted network traffic classification. It highlights that most studies utilize the ISCXVPN2016 dataset, which contains known discrepancies. Additionally, many approaches either assume that traffic can be segmented into multiple flows or rely on application payload content, neither of which is applicable to real VPN traffic scenarios.

TABLE II
CHARACTERISTICS OF OUR DATASET

Category	Application Name	Size (MB)			Duration (sec)			Total Instances (per 60 seconds sliding window)		
		Japan	England	Canada	Japan	England	Canada	Japan	England	Canada
Label 0 (Chat)	Skype_chat	76	12.8	16.5	5,437	4,833	5,406	5,378	4,774	5,347
	Whatsapp_chat	23.5	257.9	152.7	5,395	5,382	5,357	5,336	5,323	5,298
	Facebook_chat	92.3	11.6	357.6	4,768	5,404	5,288	4,709	5,345	5,229
	Telegram_chat	15.6	36.7	21	5,395	5,357	5,407	5,336	5,298	5,348
	GoogleHangouts_chat	16.6	13.1	6.8	5,169	5,373	5,315	5,110	5,314	5,256
Label 1 (Video)	Youtube	555.8	213.4	334.3	5,421	4,855	5,406	5,362	4,796	5,347
	Netflix	570.6	568.8	607.5	5,645	5,354	5,392	5,586	5,295	5,333
	Vimeo	1,021.8	649.7	474.9	4,739	5,321	5,399	4,680	5,262	5,340
Label 2 (File Transfer)	Whatsapp_files	180.5	252.2	1,130	5,315	5,287	5,351	5,256	5,228	5,292
	Dropbox	1,600	1,160	1,160	5,342	5,387	5,391	5,283	5,328	5,332
	Gdrive	1,520	1,010	1,810	4,846	5,379	5,396	4,787	5,320	5,337
	Skype_files	649.2	1,010	657.2	5,337	5,373	5,128	5,278	5,314	5,069
	Telegram_files	1,200	853.9	3,150	5,374	5,346	5,398	5,315	5,287	5,339
Label 3 (Video Conferencing)	GoogleMeets	60.7	408.3	298.7	5,415	5,384	5,407	5,356	5,325	5,348
	MicrosoftTeams	85.5	85	140.7	5,405	5,376	5,371	5,346	5,317	5,312
	Skype_video	71.1	50.3	229.7	5,241	5,380	5,400	5,182	5,321	5,341
	Zoom	319	458.5	87.7	5,406	5,077	5,354	5,347	5,018	5,295

B. Publicly Available Network Traffic Dataset

In most prior works, the ISCXVPN2016 dataset (VPN-nonVPN dataset) was used to demonstrate the effectiveness of the proposed NTC and V-NTC approaches [42], [43], [44], [45], [46], [47]. However, on taking a more in-depth look at the content of the dataset's files, we made the following observations:

- In some PCAP files, the content of the packets is in clear text.
- The port numbers used are well-known port numbers for different services.
- In some PCAP files, the protocols are TLS, HTTP, etc.
- Certain structures of the protocol (like HTML pages for the HTTP protocol) are present in the clear text.
- On resolving the DNS name for the IP address, we found that the IP address belongs to the server of well-known applications.
- There are multiple flow records.

The above observations derived from the VPN-encrypted traffic PCAP files suggest that the dataset does not contain VPN-encrypted data (see also A). Ideally, in VPN-encrypted data, we would see one (or a few) flow records for the VPN tunnel(s), which might not use well-known ports. Also, due to the presence of the VPN tunnel in real VPN-encrypted traffic, network traffic should not contain the structure/header of any protocol; however, in this dataset, protocols are detected based on the structure/header.

Since there is a need for a VPN-encrypted dataset on which NTC and V-NTC approaches can be tested, to evaluate the method proposed in this study, we created our own dataset, details of which are provided in Section III.

III. ENCRYPTED TRAFFIC DATASET

In this section, we describe the characteristics of our dataset and provide an overview of how it was created. We collected 25 GB of labeled network traffic and approximately 76.5 hours of packet capture from our four categories of traffic (chat, video, file transfer, and video conferencing), containing both VPN-encrypted and non-VPN-encrypted traffic flows. Data was collected using one VPN client, which was configured for three geographical locations (Canada, England, and Japan). The characteristics of our dataset are presented in Table II. The categories are based on those used in [16] with some exceptions (like P2P, Web browsing).

A. Dataset Characteristics

The three geographical locations (Europe, North America, and Asia) were used to observe the impact that location has on the performance of our approach. Traffic from each application was captured for 1.5 hours, thus the recorded PCAP files in the dataset are nearly uniform (+/- 5-10 minutes). The first difference observed in the data from the three locations was in the amount of traffic recorded (in GB) for a fixed period of time (1.5 hours). When using a VPN client for the geographical location of Canada, 10 GB of traffic was captured, whereas 7 GB was captured for England, and 8 GB was captured for Japan.

After capturing the traffic, the PCAP files were cleaned, removing the background noise, and segregated so they only contained the traffic flow from the VPN tunnel; this way, the dataset can be used in future research without the need to clean the files again.

B. Network Setup

To produce the dataset, we used VirtualBox to create a virtual machine on top of the host computer. Wireshark was used on both the virtual machine and the host computer to capture non-VPN and VPN-encrypted network traffic, respectively. OpenVPN was used on the host computer as the VPN client with configurations for three different countries: Japan, Canada, and England, i.e., the VPN servers were located in those locations (while the VPN client was located in Israel). Firefox was used for all of the applications requiring a Web browser. VPN-encrypted traffic was captured between the VPN client (host computer) and the VPN server. In addition, non-VPN-encrypted traffic was captured between the virtual machine and the application layer.

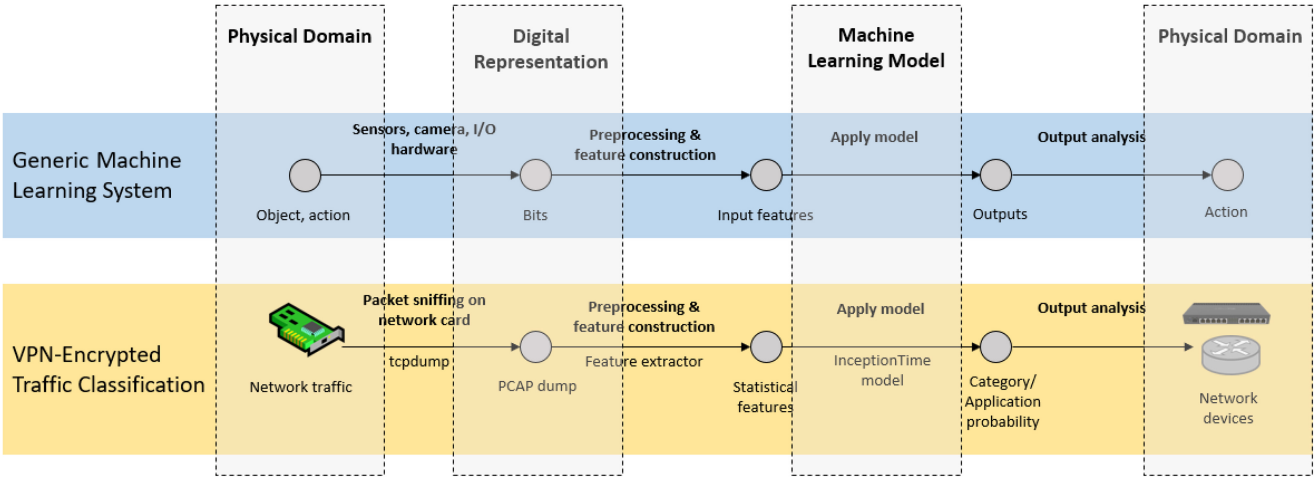


Fig. 1. Comparison of generic machine learning systems and VPN-encrypted traffic classification framework.

C. Data Generation

The dataset was made up of traffic from each application mentioned in Table II. Wireshark was used to create a single 1.5 hour PCAP file for each application. The traffic in the video category, which consists of Vimeo, Netflix, and YouTube, was produced by members of the research team as they watched videos on each platform. The traffic in the chat category, which consists of Skype, Facebook, Google Hangouts, WhatsApp, and Telegram, was created by team members as they chatted with one another on each platform. The traffic in the video conferencing category, which consists of Skype, Google Meet, Zoom, and Microsoft Teams, was generated by team members as they participated in conference calls on each platform. For the file transfer category, which consists of WhatsApp, Dropbox, Google Drive, Skype, and Telegram, we uploaded and downloaded files of various sizes on the different applications and captured the traffic.

IV. Methods

A comparison between a generic machine learning system and our VPN-encrypted traffic classification approach is provided in Fig. 1, which highlights the process flow from data capture to output analysis; shows how network traffic is captured, preprocessed, modeled, and analyzed to determine application or category probabilities; and aids in network device management. The subsections that follow provide detailed explanations of each stage: data preprocessing, feature construction, and classification model.

A. Data Preprocessing

From the PCAP file of each application, we first filtered out the single largest flow (by size) which contains the application's VPN-encrypted traffic. By performing this step we eliminated any unwanted traffic (noise) generated by other applications running in the background. The published dataset will consist of these cleaned PCAP files. After obtaining the single VPN-encrypted flow for each application, it was processed using Tshark[1] to extract fields like the timestamp, source and destination IP address, source and destination port number, and packet size of each packet. This process results in a raw feature file for each application.

[1]https://tshark.dev

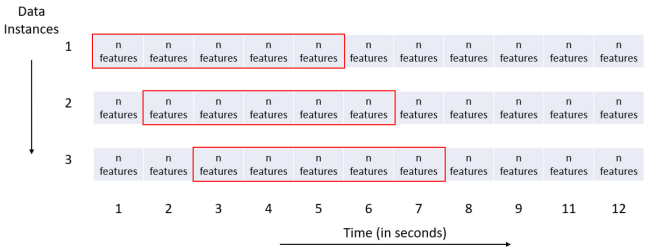


Fig. 2. Data instances construction using sliding window.

B. Feature Construction

Within each feature file, the packets are grouped, per second; then the features listed in Table III are created for each group. A one-second sliding window is used to provide high granularity and capture real-time variations in traffic, which is essential for timely detection and response. Statistical features were also included, as they were shown to be effective for online network intrusion detection in the paper presenting Kitsune [48]. We focus on three primary series of features: time delta, packet sizes, and accumulated directional packet sizes, in addition to some other features. The time delta series involves the generation of statistical features based on the time difference ratios between consecutive packets. For the packet sizes series, we create statistical features related to the size ratios between consecutive packets. In contrast, the accumulated directional packet sizes series involves the construction of statistical features that account for the ratio of cumulative packet size between consecutive packets, considering the direction of each packet. This comprehensive approach ensures that we capture both instantaneous and cumulative behaviors in packet transmission (refer to Algorithm 1 in the Appendix for the pseudocode).

Then we applied the concept of the sliding window, wherein a single instance is created by appending n features of m seconds (i.e., the window length) which results in a single instance with the shape of 1 x n x m (refer Fig. 2 where

TABLE III
DETAILED DESCRIPTIONS OF FEATURES, INCLUDING THEIR SERIES, SUB-SERIES, AND RELEVANCE TO VPN TRAFFIC

Index	Series	Sub-Series	Feature	Explanation	Relevant for VPN Traffic
1	time delta	total packets	Mean	$\mu(r^{bi}_{\Delta t})$ - The mean time delta ratio between two consecutive packets.	Yes
2			Standard deviation	$\sigma(r^{bi}_{\Delta t})$ - The standard deviation of the time delta ratio between two successive packets.	Yes
3		outgoing packets	Mean	$\mu(r^{out}_{\Delta t})$ - The mean time delta ratio between two successive outgoing packets.	Yes
4			Standard deviation	$\sigma(r^{out}_{\Delta t})$ - The standard deviation of the time delta ratio between two successive outgoing packets.	Yes
5		incoming packets	Mean	$\mu(r^{in}_{\Delta t})$ - The mean time delta ratio between two successive incoming packets.	Yes
6			Standard deviation	$\sigma(r^{in}_{\Delta t})$ - The standard deviation of the time delta ratio between two successive incoming packets.	Yes
7		2D statistics based on incoming and outgoing sub-series	Magnitude	$\sqrt{\mu(r^{in}_{\Delta t})^2 + \mu(r^{out}_{\Delta t})^2}$ - The square root of the sum of squares of the time delta ratios of incoming and outgoing packets.	Yes
8			Radius	$\sqrt{\sigma(r^{in}_{\Delta t})^2 + \sigma(r^{out}_{\Delta t})^2}$ - The square root of the sum of squares of the time delta ratios deviations of incoming and outgoing packets.	Yes
9			Covariance	$\text{Cov}(r^{in}_{\Delta t}, r^{out}_{\Delta t})$ - The time delta ratios covariance of incoming and outgoing packets.	Yes
10			Correlation coefficient	$\frac{\text{Cov}(r^{in}_{\Delta t}, r^{out}_{\Delta t})}{\sigma(r^{in}_{\Delta t}) \cdot \sigma(r^{out}_{\Delta t})}$ - The ratio between time delta ratios covariance of incoming and outgoing packets and the product of their deviations.	Yes
11	packet sizes	total packets	Mean	$\mu(r^{bi}_{size})$ - The mean packet size ratio between all packets.	Yes
12			Standard deviation	$\sigma(r^{bi}_{size})$ - The standard deviation of the packet size ratio between all packets.	Yes
13		outgoing packets	Mean	$\mu(r^{out}_{size})$ - The mean packet size ratio between the outgoing packets.	Yes
14			Standard deviation	$\sigma(r^{out}_{size})$ - The standard deviation of the packet size ratio between the outgoing packets.	Yes
15			fw_250_pkts	The number of outgoing packets of size 0-250.	Yes
16			fw_500_pkts	The number of outgoing packets of size 250-500.	Yes
17			fw_750_pkts	The number of outgoing packets of size 500-750.	Yes
18			fw_1000_pkts	The number of outgoing packets of size 750-1000.	Yes
19			fw_1250_pkts	The number of outgoing packets of size 1000-1250.	Yes
20			fw_1500_pkts	The number of outgoing packets of size 1250-1500.	Yes
21		incoming packets	Mean	$\mu(r^{in}_{size})$ - The mean packet size ratio between the incoming packets.	Yes
22			Standard deviation	$\sigma(r^{in}_{size})$ - The standard deviation of the packet size ratio between the incoming packets.	Yes
23			bw_250_pkts	The number of incoming packets of size 0-250.	Yes
24			bw_500_pkts	The number of incoming packets of size 250-500.	Yes
25			bw_750_pkts	The number of incoming packets of size 500-750.	Yes
26			bw_1000_pkts	The number of incoming packets of size 750-1000.	Yes
27			bw_1250_pkts	The number of incoming packets of size 1000-1250.	Yes
28			bw_1500_pkts	The number of incoming packets of size 1250-1500.	Yes
29		2D statistics based on incoming and outgoing sub-series	Magnitude	$\sqrt{\mu(r^{in}_{size})^2 + \mu(r^{out}_{size})^2}$ - The square root of the sum of squares of the mean size ratios of incoming and outgoing packets.	Yes
30			Radius	$\sqrt{\sigma(r^{in}_{size})^2 + \sigma(r^{out}_{size})^2}$ - The square root of the sum of squares of the size ratios deviations of incoming and outgoing packets.	Yes
31			Covariance	$\text{Cov}(r^{in}_{size}, r^{out}_{size})$ - The size ratios covariance of incoming and outgoing packets.	Yes
32			Correlation coefficient	$\frac{\text{Cov}(r^{in}_{size}, r^{out}_{size})}{\sigma(r^{in}_{size}) \cdot \sigma(r^{out}_{size})}$ - The ratio between size ratios covariance of incoming and outgoing packets and the product of their deviations.	Yes
33	accumulated directional packet sizes	total packets	Mean	$\mu(r^{bi}_{\sum size})$ - The mean accumulated packet size between all packets.	Yes
34			Standard deviation	$\sigma(r^{bi}_{\sum size})$ - The standard deviation of the accumulated packet size ratio between all packets.	Yes
35		outgoing packets	Mean	$\mu(r^{out}_{\sum size})$ - The mean accumulated outgoing packet size ratio.	Yes
36			Standard deviation	$\sigma(r^{out}_{\sum size})$ - The standard deviation of the accumulated outgoing packet size ratio.	Yes
37		incoming packets	Mean	$\mu(r^{in}_{\sum size})$ - The mean accumulated incoming packet size ratio.	Yes
38			Standard deviation	$\sigma(r^{in}_{\sum size})$ - The standard deviation of the accumulated incoming packet size ratio.	Yes
39		2D statistics based on incoming and outgoing sub-series	Magnitude	$\sqrt{\mu(r^{in}_{\sum size})^2 + \mu(r^{out}_{\sum size})^2}$ - The square root of the sum of squares of the mean accumulated size ratios of incoming and outgoing packets.	Yes
40			Radius	$\sqrt{\sigma(r^{in}_{\sum size})^2 + \sigma(r^{out}_{\sum size})^2}$ - The square root of the sum of squares of the accumulated size ratios deviations of incoming and outgoing packets.	Yes
41			Covariance	$\text{Cov}(r^{in}_{\sum size}, r^{out}_{\sum size})$ - The accumulated size ratios covariance of incoming and outgoing packets.	Yes
42			Correlation coefficient	$\frac{\text{Cov}(r^{in}_{\sum size}, r^{out}_{\sum size})}{\sigma(r^{in}_{\sum size}) \cdot \sigma(r^{out}_{\sum size})}$ - The ratio between accumulated size ratios covariance of incoming and outgoing packets and the product of their deviations.	Yes
43	additional features	___	total_frame_len	The total frame length.	Yes
44			uniq_dst_IPs	The number of unique destination ips.	No
45			uniq_src_IPs	The number of unique source ips.	No
46			unique_dst_ports	The number of unique destination ports.	No
47			unique_src_ports	The number of unique source ports.	No

the window length is five). Table II contains the number of data instances (column *Total Instances*) after the features were constructed. We also performed standardization (i.e., Z-score normalization) on the entire dataset, transforming the data so that it has a mean of zero and a standard deviation of one. Our aim in performing this feature construction

TABLE IV
SUMMARY OF THE EXPERIMENTS PERFORMED

Experiment Number and Name	Total Labels	Network Traffic Considered	VPN-Encrypted Traffic	Description
1. Application category classification	4	Entire dataset	Yes	Traffic from all three countries was combined with the respective application traffic, and classification was performed at the category level.
2. Application traffic classification	17	Entire dataset	Yes	Traffic from all three countries was combined with the respective application traffic, and classification was performed at the application level.
3. Application category classification by each country	4	Single Country	Yes	Traffic from a single country was considered (i.e., either Canada, England, or Japan) and classification was performed at the category level.
4. Application traffic classification by each country	17	Single Country	Yes	Traffic from a single country was considered (i.e., either Canada, England, or Japan), and classification was performed at the application level.
5. Application category classification	6	Subset of ISCXVPN2016 dataset	No	Non-VPN traffic from the ISCXVPN2016 dataset was used, and classification was performed at the category level.
6. Application traffic classification	21	Subset of ISCXVPN2016 dataset	No	Non-VPN traffic from the ISCXVPN2016 dataset was used, and classification was performed at the application level.
7. IoT device classification	11 & 9	Subset of IoT Trace dataset	No	Two models were trained, and traffic from the IoT devices listed in Tables XI and XII was considered to identify the IoT device type.

process was to convert the network traffic data to time-series data.

C. Classification Model

For the classification of the VPN-encrypted network traffic (that was converted to time-series data), we used the InceptionTime model [15]. This model is an inception-based network [49] that applies several convolutions with filters of various lengths. It consists of an ensemble of five different inception networks that are initialized randomly.

Inception network classifiers contain two different residual blocks. For the inception network, each block is comprised of three inception modules rather than traditional fully convolutional layers. Each inception module applies convolutions with lengths of 10, 20, and 40, with a max pooling operation followed by a bottleneck layer, and concatenates the outputs. Each residual block's input is transferred via a shortcut linear connection so it can be added to the next block's input, thus mitigating the vanishing gradient problem by allowing a direct flow of the gradient [50]. The padding chosen aims to maintain the dimensionality of the time series after the convolutions. After the residual blocks, a global average pooling (GAP) layer is employed which averages the output (in the form of multivariate time series) over the whole time dimension. The network is comprised of six inception modules stacked sequentially within each residual block. Each module includes a bottleneck layer to reduce dimensionality, which is followed by multiple convolutional layers with different filter lengths. Then, a final traditional fully connected softmax layer is used, with the number of neurons equal to the number of classes in the dataset. Fig. 3 depicts the architecture of an inception network in which six inception modules are stacked one after the other. The inception modules use multiple filters of varying lengths, specifically three sets of filters each with lengths of 10, 20, and 40. For more details about the model, please refer to [15].

V. EXPERIMENTS AND RESULTS

Table IV provides a summary of the experiments performed. We divided the data into training and test sets for each

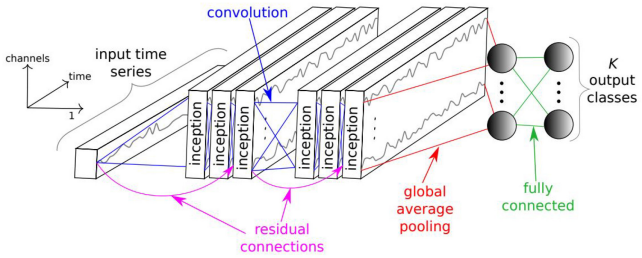


Fig. 3. Inception network for time-series classification [49].

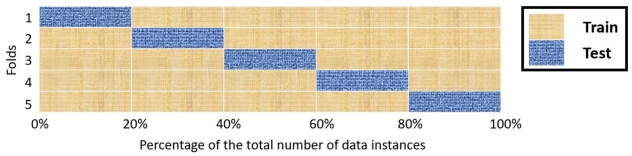


Fig. 4. Data split for 5-fold cross-validation in time-series analysis.

application, as shown in Fig. 4, for five folds, where 80% of the dataset was used for training, and 20% was used for testing. We divided the training and test set in a sequential manner to avoid leaking any information resulting from the features' construction (described in Section IV-B) to the test set. We used a batch size of 64, and training was performed for 10 epochs. The metrics used in our evaluation are the F1-score and accuracy. Note that as shown in Table III, feature numbers 44-47 will not have any impact in experiments 1-4, because in VPN traffic, the unique source and destination IP and the unique source and destination port will remain the same across all applications. Values for these features will differ in experiments 5-6 and therefore will have an impact on the performance in those cases.

Experiments 1-2: Classification of Network Traffic at the Category- and Application-Level for the Entire Dataset

In these two experiments, traffic from all three geographical locations was combined. In the first experiment, there were four labels, i.e., four application categories. The classification accuracy and F1-score results are presented in Table V; as can be seen, when the length of the sliding window was

TABLE V
EXPERIMENT 1 - PERFORMANCE OF DIFFERENT APPLICATIONS FOR CATEGORY-LEVEL CLASSIFICATION WITH A SLIDING WINDOW LENGTH OF 60 SECONDS

		Japan		England		Canada	
Category Name	**Application Name**	**Accuracy**	**F1-Score**	**Accuracy**	**F1-Score**	**Accuracy**	**F1-Score**
Label 0 (Chat)	Skype_chat	0.966 +/- 0.052	0.982 +/- 0.028	0.955 +/- 0.017	0.977 +/- 0.009	0.994 +/- 0.005	0.997 +/- 0.002
	Whatsapp_chat	0.941 +/- 0.062	0.969 +/- 0.034	0.804 +/- 0.104	0.888 +/- 0.064	0.876 +/- 0.121	0.929 +/- 0.076
	Facebook_chat	1.0 +/- 0.0	1.0 +/- 0.0	0.999 +/- 0.002	1.0 +/- 0.0	0.853 +/- 0.137	0.914 +/- 0.083
	Telegram_chat	0.942 +/- 0.046	0.969 +/- 0.025	0.945 +/- 0.051	0.971 +/- 0.028	0.953 +/- 0.043	0.975 +/- 0.023
	GoogleHangouts_chat	0.967 +/- 0.034	0.983 +/- 0.018	0.997 +/- 0.005	0.999 +/- 0.002	0.972 +/- 0.024	0.985 +/- 0.012
	Overall for Label 0	**0.963 +/- 0.038**	**0.98 +/- 0.021**	**0.94 +/- 0.035**	**0.967 +/- 0.02**	**0.929 +/- 0.066**	**0.96 +/- 0.039**
Label 1 (Video)	Youtube	0.902 +/- 0.052	0.948 +/- 0.029	0.912 +/- 0.065	0.953 +/- 0.036	0.789 +/- 0.127	0.876 +/- 0.084
	Netflix	0.987 +/- 0.026	0.993 +/- 0.014	0.95 +/- 0.035	0.974 +/- 0.019	1.0 +/- 0.0	1.0 +/- 0.0
	Vimeo	0.842 +/- 0.256	0.887 +/- 0.195	0.989 +/- 0.019	0.995 +/- 0.009	0.982 +/- 0.028	0.991 +/- 0.015
	Overall for Label 1	**0.91 +/- 0.111**	**0.942 +/- 0.079**	**0.95 +/- 0.039**	**0.974 +/- 0.021**	**0.923 +/- 0.051**	**0.955 +/- 0.033**
Label 2 (File Transfer)	Whatsapp_files	0.763 +/- 0.207	0.85 +/- 0.137	0.435 +/- 0.165	0.588 +/- 0.161	0.786 +/- 0.281	0.846 +/- 0.215
	Dropbox	0.968 +/- 0.057	0.983 +/- 0.031	0.953 +/- 0.05	0.975 +/- 0.027	0.957 +/- 0.086	0.976 +/- 0.048
	Gdrive	1.0 +/- 0.0	1.0 +/- 0.0	0.967 +/- 0.052	0.983 +/- 0.028	1.0 +/- 0.0	1.0 +/- 0.0
	Skype_files	0.874 +/- 0.139	0.927 +/- 0.083	1.0 +/- 0.0	1.0 +/- 0.0	0.964 +/- 0.062	0.98 +/- 0.034
	Telegram_files	0.924 +/- 0.151	0.953 +/- 0.093	0.91 +/- 0.121	0.948 +/- 0.07	0.984 +/- 0.02	0.992 +/- 0.01
	Overall for Label 2	**0.905 +/- 0.11**	**0.942 +/- 0.068**	**0.853 +/- 0.077**	**0.898 +/- 0.057**	**0.938 +/- 0.0.089**	**0.958 +/- 0.061**
Label 3 (Video Conferencing)	GoogleMeets	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
	MicrosoftTeams	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
	Skype_video	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	0.926 +/- 0.096	0.959 +/- 0.054
	Zoom	0.92 +/- 0.156	0.951 +/- 0.097	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
	Overall for Label 3	**0.98 +/- 0.039**	**0.987 +/- 0.024**	**1.0 +/- 0.0**	**1.0 +/- 0.0**	**0.981 +/- 0.024**	**0.989 +/- 0.013**
	Total	**Accuracy: 0.938 +/- 0.012 and F1-Score: 0.938 +/- 0.012**					

TABLE VI
EXPERIMENT 2 - PERFORMANCE OF DIFFERENT APPLICATIONS FOR APPLICATION-LEVEL CLASSIFICATION WITH A SLIDING WINDOW LENGTH OF 60 SECONDS

	Japan		England		Canada	
Application Name	**Accuracy**	**F1-Score**	**Accuracy**	**F1-Score**	**Accuracy**	**F1-Score**
Skype_chat	0.872 +/- 0.082	0.93 +/- 0.048	0.874 +/- 0.06	0.931 +/- 0.034	0.896 +/- 0.041	0.944 +/- 0.022
Whatsapp_chat	0.303 +/- 0.099	0.455 +/- 0.127	0.55 +/- 0.18	0.692 +/- 0.15	0.648 +/- 0.101	0.782 +/- 0.073
Facebook_chat	0.992 +/- 0.016	0.996 +/- 0.008	0.961 +/- 0.035	0.98 +/- 0.018	0.891 +/- 0.13	0.937 +/- 0.076
Telegram_chat	0.609 +/- 0.079	0.754 +/- 0.064	0.526 +/- 0.168	0.673 +/- 0.153	0.774 +/- 0.049	0.872 +/- 0.03
GoogleHangouts_chat	0.632 +/- 0.109	0.769 +/- 0.089	0.833 +/- 0.071	0.907 +/- 0.045	0.71 +/- 0.165	0.818 +/- 0.123
Youtube	0.894 +/- 0.059	0.943 +/- 0.034	0.892 +/- 0.093	0.94 +/- 0.054	0.847 +/- 0.066	0.916 +/- 0.04
Netflix	0.981 +/- 0.03	0.99 +/- 0.016	0.96 +/- 0.038	0.979 +/- 0.02	0.999 +/- 0.002	1.0 +/- 0.0
Vimeo	0.82 +/- 0.244	0.877 +/- 0.185	0.991 +/- 0.018	0.996 +/- 0.009	0.97 +/- 0.03	0.985 +/- 0.016
Whatsapp_files	0.627 +/- 0.391	0.684 +/- 0.364	0.589 +/- 0.069	0.739 +/- 0.056	0.759 +/- 0.177	0.851 +/- 0.116
Dropbox	0.946 +/- 0.094	0.97 +/- 0.054	0.97 +/- 0.037	0.984 +/- 0.019	0.82 +/- 0.142	0.894 +/- 0.087
Gdrive	1.0 +/- 0.0	1.0 +/- 0.0	0.958 +/- 0.039	0.978 +/- 0.02	0.971 +/- 0.059	0.984 +/- 0.032
Skype_files	0.75 +/- 0.165	0.846 +/- 0.118	0.917 +/- 0.116	0.953 +/- 0.068	0.9 +/- 0.124	0.942 +/- 0.071
Telegram_files	0.975 +/- 0.042	0.987 +/- 0.023	0.833 +/- 0.178	0.898 +/- 0.117	0.94 +/- 0.077	0.967 +/- 0.042
GoogleMeets	0.996 +/- 0.008	0.998 +/- 0.004	0.999 +/- 0.001	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
MicrosoftTeams	0.995 +/- 0.009	0.998 +/- 0.004	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
Skype_video	0.962 +/- 0.045	0.98 +/- 0.024	0.999 +/- 0.003	0.999 +/- 0.001	0.871 +/- 0.158	0.923 +/- 0.094
Zoom	0.93 +/- 0.14	0.958 +/- 0.085	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
Total	**Accuracy: 0.865 +/- 0.008 and F1-Score: 0.865 +/- 0.008**					

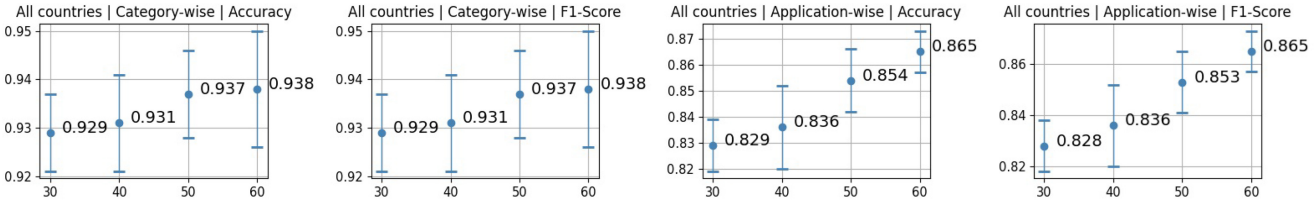


Fig. 5. Performance of the models trained on the entire dataset for various sliding window lengths (Y-axis).

60 seconds, the overall accuracy obtained was 93.80%. In the second experiment, there were 17 labels, i.e., one for each application. The classification accuracy and F1-score results are presented in Table VI; as can be seen, when the length of the sliding window was 60 seconds, the overall accuracy obtained was 86.50%. Note that to obtain a granular view of the performance, we examined each application's test data against the model; the results can be found in Tables V and VI. The results for smaller sized sliding windows are presented in Fig. 5; results for longer sized sliding windows are presented in Fig. 8 in the Appendix. As can be seen, larger sliding windows, such as 120 seconds, result in minimal improvement in accuracy while significantly increasing latency. Based on this analysis, a 60-second sliding window was chosen as it

TABLE VII
EXPERIMENT 3 - PERFORMANCE OF DIFFERENT APPLICATIONS FOR EACH COUNTRY FOR CATEGORY-LEVEL CLASSIFICATION WITH A SLIDING WINDOW LENGTH OF 60 SECONDS

		Japan		England		Canada	
Category Name	**Application Name**	**Accuracy**	**F1-Score**	**Accuracy**	**F1-Score**	**Accuracy**	**F1-Score**
Label 0 (Chat)	Skype_chat	0.96 +/- 0.052	0.979 +/- 0.028	0.953 +/- 0.02	0.976 +/- 0.011	0.996 +/- 0.008	0.998 +/- 0.004
	Whatsapp_chat	0.924 +/- 0.08	0.959 +/- 0.045	0.91 +/- 0.096	0.95 +/- 0.054	0.833 +/- 0.111	0.905 +/- 0.07
	Facebook_chat	1.0 +/- 0.0	1.0 +/- 0.0	0.988 +/- 0.016	0.994 +/- 0.008	0.93 +/- 0.056	0.963 +/- 0.03
	Telegram_chat	0.966 +/- 0.023	0.983 +/- 0.012	0.988 +/- 0.01	0.994 +/- 0.005	0.978 +/- 0.022	0.989 +/- 0.012
	GoogleHangouts_chat	0.984 +/- 0.014	0.992 +/- 0.007	0.981 +/- 0.013	0.99 +/- 0.007	0.98 +/- 0.02	0.99 +/- 0.01
	Overall for Label 0	**0.968 +/- 0.033**	**0.982 +/- 0.018**	**0.964 +/- 0.031**	**0.98 +/- 0.017**	**0.943 +/- 0.043**	**0.969 +/- 0.012**
Label 1 (Video)	Youtube	0.925 +/- 0.03	0.961 +/- 0.017	0.863 +/- 0.102	0.923 +/- 0.061	0.798 +/- 0.085	0.885 +/- 0.054
	Netflix	1.0 +/- 0.0	1.0 +/- 0.0	0.975 +/- 0.024	0.987 +/- 0.012	1.0 +/- 0.0	1.0 +/- 0.0
	Vimeo	0.851 +/- 0.254	0.893 +/- 0.191	0.977 +/- 0.036	0.988 +/- 0.019	0.975 +/- 0.039	0.987 +/- 0.021
	Overall for Label 1	**0.925 +/- 0.094**	**0.951 +/- 0.069**	**0.938 +/- 0.054**	**0.966 +/- 0.03**	**0.924 +/- 0.041**	**0.957 +/- 0.025**
Label 2 (File Transfer)	Whatsapp_files	0.78 +/- 0.284	0.843 +/- 0.212	0.702 +/- 0.157	0.814 +/- 0.116	0.704 +/- 0.192	0.811 +/- 0.133
	Dropbox	0.961 +/- 0.062	0.979 +/- 0.034	0.946 +/- 0.054	0.972 +/- 0.029	0.961 +/- 0.072	0.979 +/- 0.04
	Gdrive	1.0 +/- 0.0	1.0 +/- 0.0	0.98 +/- 0.027	0.989 +/- 0.014	0.998 +/- 0.004	0.999 +/- 0.002
	Skype_files	0.866 +/- 0.143	0.922 +/- 0.086	0.993 +/- 0.013	0.997 +/- 0.007	0.943 +/- 0.087	0.968 +/- 0.049
	Telegram_files	0.986 +/- 0.017	0.993 +/- 0.009	0.993 +/- 0.014	0.996 +/- 0.007	0.971 +/- 0.058	0.984 +/- 0.031
	Overall for Label 2	**0.918 +/- 0.101**	**0.947 +/- 0.098**	**0.922 +/- 0.053**	**0.953 +/- 0.034**	**0.915 +/- 0.082**	**0.948 +/- 0.051**
Label 3 (Video Conferencing)	GoogleMeets	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
	MicrosoftTeams	0.998 +/- 0.005	0.999 +/- 0.002	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
	Skype_video	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	0.946 +/- 0.108	0.969 +/- 0.063
	Zoom	0.933 +/- 0.134	0.96 +/- 0.081	0.994 +/- 0.011	0.997 +/- 0.006	1.0 +/- 0.0	1.0 +/- 0.0
	Overall for Label 3	**0.982 +/- 0.034**	**0.989 +/- 0.02**	**0.998 +/- 0.002**	**0.999 +/- 0.001**	**0.986 +/- 0.027**	**0.992 +/- 0.015**
	Total	**0.949 +/- 0.022**	**0.956 +/- 0.018**	**0.956 +/- 0.018**	**0.949 +/- 0.022**	**0.942 +/- 0.03**	**0.942 +/- 0.029**

TABLE VIII
EXPERIMENT 4 - PERFORMANCE OF DIFFERENT APPLICATIONS FOR EACH COUNTRY FOR APPLICATION-LEVEL CLASSIFICATION FOR A SLIDING WINDOW LENGTH OF 60 SECONDS

	Japan		England		Canada	
Application Name	**Accuracy**	**F1-Score**	**Accuracy**	**F1-Score**	**Accuracy**	**F1-Score**
Skype_chat	0.866 +/- 0.118	0.924 +/- 0.071	0.91 +/- 0.024	0.952 +/- 0.013	0.924 +/- 0.039	0.96 +/- 0.022
Whatsapp_chat	0.402 +/- 0.079	0.569 +/- 0.082	0.608 +/- 0.167	0.743 +/- 0.136	0.702 +/- 0.149	0.816 +/- 0.104
Facebook_chat	0.987 +/- 0.017	0.993 +/- 0.009	0.982 +/- 0.02	0.991 +/- 0.01	0.923 +/- 0.084	0.958 +/- 0.048
Telegram_chat	0.596 +/- 0.114	0.74 +/- 0.087	0.712 +/- 0.163	0.82 +/- 0.122	0.825 +/- 0.042	0.903 +/- 0.026
GoogleHangouts_chat	0.691 +/- 0.14	0.809 +/- 0.099	0.87 +/- 0.104	0.927 +/- 0.063	0.73 +/- 0.157	0.833 +/- 0.119
Youtube	0.934 +/- 0.025	0.966 +/- 0.013	0.877 +/- 0.072	0.933 +/- 0.04	0.858 +/- 0.087	0.921 +/- 0.051
Netflix	0.956 +/- 0.081	0.976 +/- 0.045	0.991 +/- 0.014	0.995 +/- 0.007	1.0 +/- 0.0	1.0 +/- 0.0
Vimeo	0.831 +/- 0.248	0.882 +/- 0.189	0.991 +/- 0.018	0.995 +/- 0.009	0.975 +/- 0.037	0.987 +/- 0.02
Whatsapp_files	0.659 +/- 0.371	0.72 +/- 0.336	0.82 +/- 0.098	0.898 +/- 0.059	0.657 +/- 0.209	0.773 +/- 0.16
Dropbox	0.849 +/- 0.137	0.912 +/- 0.082	0.966 +/- 0.039	0.982 +/- 0.021	0.896 +/- 0.135	0.94 +/- 0.081
Gdrive	1.0 +/- 0.0	1.0 +/- 0.0	0.972 +/- 0.032	0.986 +/- 0.016	0.963 +/- 0.072	0.98 +/- 0.039
Skype_files	0.762 +/- 0.166	0.854 +/- 0.114	0.971 +/- 0.058	0.984 +/- 0.031	0.937 +/- 0.076	0.966 +/- 0.042
Telegram_files	0.967 +/- 0.042	0.982 +/- 0.022	0.898 +/- 0.199	0.932 +/- 0.133	0.888 +/- 0.196	0.927 +/- 0.132
GoogleMeets	1.0 +/- 0.0	1.0 +/- 0.0	0.918 +/- 0.165	0.948 +/- 0.104	1.0 +/- 0.0	1.0 +/- 0.0
MicrosoftTeams	0.994 +/- 0.012	0.997 +/- 0.006	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
Skype_video	0.955 +/- 0.048	0.976 +/- 0.026	1.0 +/- 0.0	1.0 +/- 0.0	0.946 +/- 0.108	0.969 +/- 0.062
Zoom	0.929 +/- 0.112	0.959 +/- 0.065	0.992 +/- 0.017	0.996 +/- 0.008	1.0 +/- 0.0	1.0 +/- 0.0
Total	**0.845 +/- 0.035**	**0.84 +/- 0.037**	**0.91 +/- 0.027**	**0.909 +/- 0.028**	**0.896 +/- 0.027**	**0.893 +/- 0.025**

strikes a balance between capturing sufficient data for accurate classification and maintaining low latency for practical real-world application. It can be seen that poorer accuracy was obtained for the *Whatsapp_files* application at the category level (see Table V) with the combined data from all three countries; this could stem from the different sizes of the files that were used when capturing the network traffic. After removing the *Whatsapp_files* application's data from the training and test samples, the overall category-level accuracy results improved from 93.80% to 96.20%, and the overall application-level accuracy results improved from 86.50% to 88.60%.

The experiments were conducted on a system with the following configuration: a single NVIDIA TITAN X (Pascal) GPU, 6 CPU cores, 125.21 GB of RAM, and 12.0 GB of GPU memory. The average CPU utilization during the experiments was 8.37%, the average GPU utilization was 4.69%, and the average RAM utilization was 11.56%. The average prediction time for 1,000 instances was 0.0639 seconds, and the training time per epoch was approximately 38 seconds.

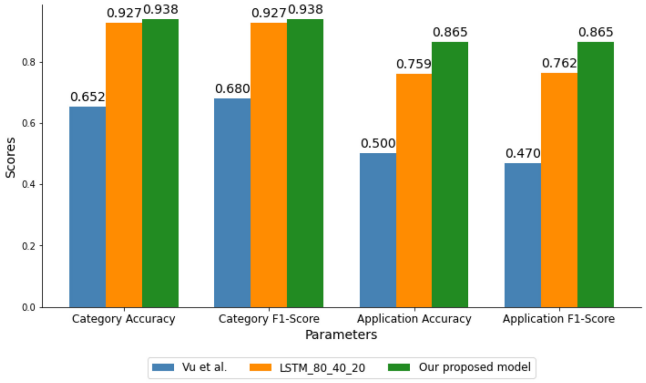


Fig. 6. Comparative analysis of various models for experiment 1-2.

TABLE IX
CHARACTERISTICS OF THE ISCXVPN2016 DATASET SUBSET

Category Name	Application Name	Duration (sec)	Number of Samples (60 seconds)
Label 0 (Email)	Email	12,452	12,216
Label 1 (Chat)	Aim	1,658	1,481
	Facebook_chat	1,246	1,128
	Skype_chat	1,925	1,807
	GoogleHangouts_chat	1,341	1,223
	ICQchat	1,779	1,602
	Gmailchat	689	630
Label 2 (Streaming)	Vimeo	1,317	1,140
	Youtube	2,419	2,183
	Netflix	825	707
	Spotify	3,875	3,639
Label 3 (File transfer)	Skype_file	7,647	7,411
	Scp	552	493
	Ftps	1,449	1,331
Label 4 (VoIP)	Facebook_audio	13,843	13,489
	Skype_audio	7,455	7,101
	GoogleHangouts_audio	13,966	13,6129
	VoipBuster	8,344	8,049
Label 5 (Video call)	Facebook_video	940	822
	GoogleHangouts_video	3,663	3,486
	Skype_video	3,246	3,010
	Total	**90,630**	**86,560**

TABLE X
EXPERIMENTS 5 & 6 - PERFORMANCE OF DIFFERENT APPLICATIONS FOR CATEGORY- AND APPLICATION-LEVEL CLASSIFICATION

		Category-Level		Application-Level	
Category Name	**Application Name**	**Accuracy**	**F1-Score**	**Accuracy**	**F1-Score**
Label 0 (Email)	Email	0.948 +/- 0.072	0.969 +/- 0.044	0.953 +/- 0.089	0.97 +/- 0.057
Label 1 (Chat)	Aim	0.98 +/- 0.04	0.988 +/- 0.023	0.584 +/- 0.258	0.641 +/- 0.269
	Facebook_chat	0.956 +/- 0.088	0.972 +/- 0.057	0.931 +/- 0.123	0.96 +/- 0.073
	Skype_chat	0.962 +/- 0.05	0.979 +/- 0.028	0.964 +/- 0.056	0.979 +/- 0.033
	GoogleHangouts_chat	0.997 +/- 0.006	0.998 +/- 0.003	0.976 +/- 0.048	0.987 +/- 0.027
	ICQchat	0.98 +/- 0.027	0.989 +/- 0.014	0.552 +/- 0.202	0.632 +/- 0.218
	Gmailchat	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
	Overall for Label 1	**0.977 +/- 0.038**	**0.986 +/- 0.022**	**0.809 +/- 0.128**	**0.845 +/- 0.117**
Label 2 (Streaming)	Vimeo	1.0 +/- 0.0	1.0 +/- 0.0	0.951 +/- 0.099	0.951 +/- 0.099
	Youtube	0.974 +/- 0.052	0.983 +/- 0.034	0.722 +/- 0.343	0.773 +/- 0.304
	Netflix	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
	Spotify	0.851 +/- 0.16	0.907 +/- 0.109	0.709 +/- 0.168	0.799 +/- 0.148
	Overall for Label 2	**0.922 +/- 0.09**	**0.951 +/- 0.061**	**0.776 +/- 0.192**	**0.833 +/- 0.171**
Label 3 (File transfer)	Skype_file	0.931 +/- 0.092	0.954+/- 0.065	0.927 +/- 0.101	0.95 +/- 0.073
	Scp	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
	Ftps	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
	Overall for Label 3	**0.945 +/- 0.074**	**0.963 +/- 0.052**	**0.941 +/- 0.081**	**0.96 +/- 0.059**
Label 4 (VoIP)	Facebook_audio	0.993 +/- 0.014	0.994 +/- 0.012	0.967 +/- 0.044	0.978 +/- 0.033
	Skype_audio	0.988 +/- 0.023	0.992 +/- 0.016	0.898 +/- 0.073	0.935 +/- 0.053
	GoogleHangouts_audio	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
	VoipBuster	0.998 +/- 0.003	0.999 +/- 0.002	0.998 +/- 0.003	0.999 +/- 0.002
	Overall for Label 4	**0.996 +/- 0.009**	**0.997 +/- 0.007**	**0.972 +/- 0.027**	**0.982 +/- 0.02**
Label 5 (Video call)	Facebook_video	0.954 +/- 0.091	0.971 +/- 0.059	0.895 +/- 0.21	0.895 +/- 0.21
	GoogleHangouts_video	0.967 +/- 0.066	0.975 +/- 0.049	0.969 +/- 0.063	0.977 +/- 0.046
	Skype_video	0.988 +/- 0.024	0.993 +/- 0.014	0.99 +/- 0.019	0.994 +/- 0.011
	Overall for Label 5	**0.974 +/- 0.052**	**0.982+/- 0.036**	**0.969 +/- 0.061**	**0.975 +/- 0.05**
	Total	**0.973 +/- 0.038**	**0.983 +/- 0.026**	**0.934 +/- 0.068**	**0.952 +/- 0.054**

The bar chart in Fig. 6 compares our proposed method's performance with that of the method of Vu et al. [29] and a method with baseline LSTM model. As can be seen, our proposed method outperforms the others on all metrics. The significant difference in application-level performance indicates that our method captures granular patterns more effectively than the other two method, reflecting its superior capability in detailed pattern recognition.

The baseline LSTM model (denoted as LSTM 80 40 20) used for comparison consists of a sequential architecture with an 80-neuron LSTM layer, followed by a 40-neuron LSTM layer, and a 20-neuron dense layer. This architecture also includes three dropout layers, each with a dropout rate of 0.5, and uses the features proposed in this paper. The worst performance of the three was obtained by the model of Vu et al. [29], as depicted in the graph.

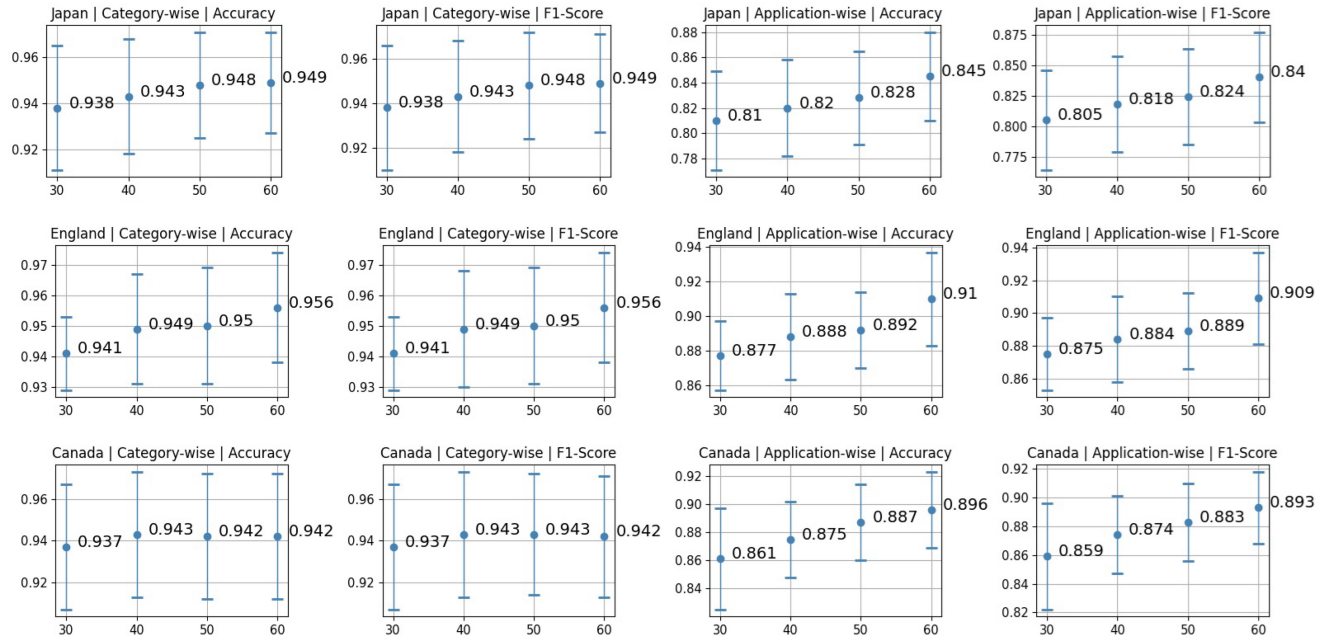


Fig. 7. Performance of the models trained on country-specific data for various sliding window lengths (Y-axis).

TABLE XI
EXPERIMENT 7 - PERFORMANCE ON IOT DATA FOR SLIDING WINDOWS OF VARIOUS LENGTHS (OCTOBER 23)

Window Length	**30 Seconds**		**40 Seconds**		**50 Seconds**		**60 Seconds**	
IoT Device Name	**Accuracy**	**F1-Score**	**Accuracy**	**F1-Score**	**Accuracy**	**F1-Score**	**Accuracy**	F1-Score
Samsung_SmartCam	0.998 +/- 0.003	0.999 +/- 0.002	1.0 +/- 0.0	1.0 +/- 0.0	0.999 +/- 0.002	0.999 +/- 0.001	1.0 +/- 0.0	1.0 +/- 0.0
Withings_Smart_Baby_Monitor	0.996 +/- 0.006	0.998 +/- 0.003	0.954 +/- 0.092	0.974 +/- 0.052	0.956 +/- 0.088	0.975 +/- 0.05	0.958 +/- 0.084	0.976 +/- 0.047
Samsung_Galaxy_Tab	0.78 +/ -0.02	0.876 +/- 0.012	0.895 +/- 0.025	0.944 +/- 0.014	0.927 +/- 0.038	0.962 +/- 0.021	0.97 +/- 0.021	0.984 +/- 0.011
Dropcam	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
Amazon_Echo	0.987 +/- 0.005	0.994 +/- 0.003	0.995 +/- 0.006	0.997 +/- 0.003	0.994 +/- 0.008	0.997 +/- 0.004	0.998 +/- 0.003	0.999 +/- 0.002
Netatmo_weather_station	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
Netatmo_Welcome	0.85 +/- 0.222	0.9 +/- 0.16	0.857 +/- 0.239	0.9 +/- 0.175	0.938 +/- 0.121	0.963 +/- 0.071	0.881 +/- 0.226	0.918 +/- 0.159
Laptop	0.979 +/- 0.016	0.989 +/- 0.008	0.987 +/- 0.015	0.993 +/- 0.008	0.992 +/- 0.01	0.996 +/- 0.005	0.993 +/- 0.012	0.996 +/- 0.006
Smart_Things	0.997 +/- 0.003	0.999 +/- 0.002	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
Total	**0.954 +/- 0.023**	**0.952 +/- 0.025**	**0.965 +/- 0.028**	**0.964 +/- 0.03**	**0.978 +/- 0.018**	**0.978 +/- 0.018**	**0.978 +/- 0.026**	**0.977 +/- 0.028**

TABLE XII
EXPERIMENT 7 - PERFORMANCE ON IOT DATA FOR SLIDING WINDOWS OF VARIOUS LENGTHS (OCTOBER 24)

Window Length	**30 Seconds**		**40 Seconds**		**50 Seconds**		**60 Seconds**	
IoT Device Name	**Accuracy**	**F1-Score**	**Accuracy**	**F1-Score**	**Accuracy**	**F1-Score**	**Accuracy**	**F1-Score**
Samsung_SmartCam	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
Withings_Smart_Baby_Monitor	0.999 +/- 0.001	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
Dropcam	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
Amazon_Echo	0.998 +/- 0.003	0.999 +/- 0.002	0.999 +/- 0.002	0.999 +/- 0.001	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
Netatmo_weather_station	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
Netatmo_Welcome	0.985 +/- 0.011	0.992 +/- 0.006	0.988 +/- 0.014	0.994 +/- 0.007	0.994 +/- 0.012	0.997 +/- 0.006	0.989 +/- 0.014	0.995 +/- 0.007
Smart_Things	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0	1.0 +/- 0.0
Total	**0.998 +/- 0.002**	**0.998 +/- 0.002**	**0.998 +/- 0.002**	**0.998 +/- 0.002**	**0.999 +/- 0.002**	**0.999 +/- 0.002**	**0.998 +/- 0.002**	**0.998 +/- 0.002**

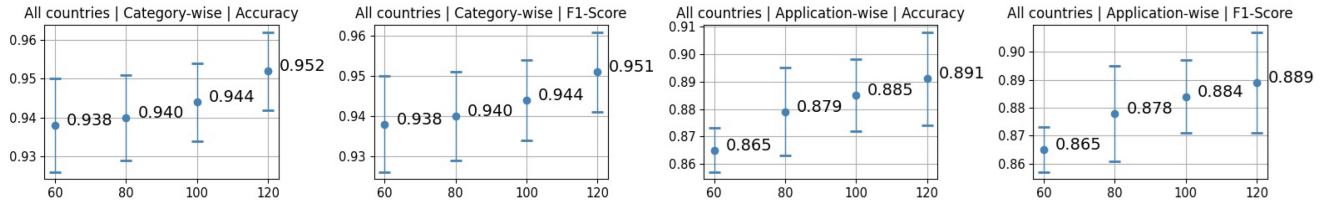


Fig. 8. Performance of the models trained on the entire dataset for prolonged sliding window lengths (Y-axis).

Experiments 3-4: Classification of Network Traffic at the Category- and Application-Level for Each Country

In these two experiments, models were trained separately for each country, i.e., trained and tested on one country. In the third experiment, there were four labels, i.e., four application categories. The classification accuracy and F1-score results for the three models trained respectively on traffic from Japan, Canada, and England are presented in Table VII. In the fourth experiment, there were 17 labels, i.e., one for each application. The classification accuracy and F1-score results

Fig. 9. HTTP protocol tags are visible in the payload of data.

for the three models trained on traffic from Japan, Canada, and England respectively are presented in Table VIII. The results for smaller sliding windows are presented in Fig. 7. As can be seen, in most cases the accuracy and F1-score results improve as the size of the sliding window increases.

Experiments 5-6: Classification of Network Traffic at the Category- and Application-Level for Non-VPN-Encrypted Traffic

In these two experiments, a subset of non-VPN traffic from the ISCXVPN2016 datset was used; the characteristics of the subset are presented in Table IX. In both experiments, we considered files in which the traffic was present for at least 300 seconds, i.e., at least five minutes of recording. Please note: In these two experiments, as there were multiple files per application, we calculated weighted accuracy. Also, as mentioned earlier, we use all the features mentioned in Table III in these experiments.

In the fifth experiment, there were six labels, i.e., six application categories. The classification accuracy and F1-score results are presented in Table X; as can be seen, the overall accuracy was 97.30% when the length of the sliding window was 60 seconds. As seen in the table, the accuracy of our approach at the category level was lower for *Spotify*; this could be due to Spotify's streaming format (audio) which differs from that of the other recorded streaming services (video).

In the sixth experiment, there were 21 labels, i.e., one for each application. The classification accuracy and F1-score results are presented in Table X; as can be seen, the overall accuracy was 93.40% when the length of the sliding window was 60 seconds.

To see the impact of the last four features (i.e., feature numbers 44 to 47), we reran these two experiments without those four features and obtained 95.5% (compared to 97.30%) accuracy in category classification (experiment five)

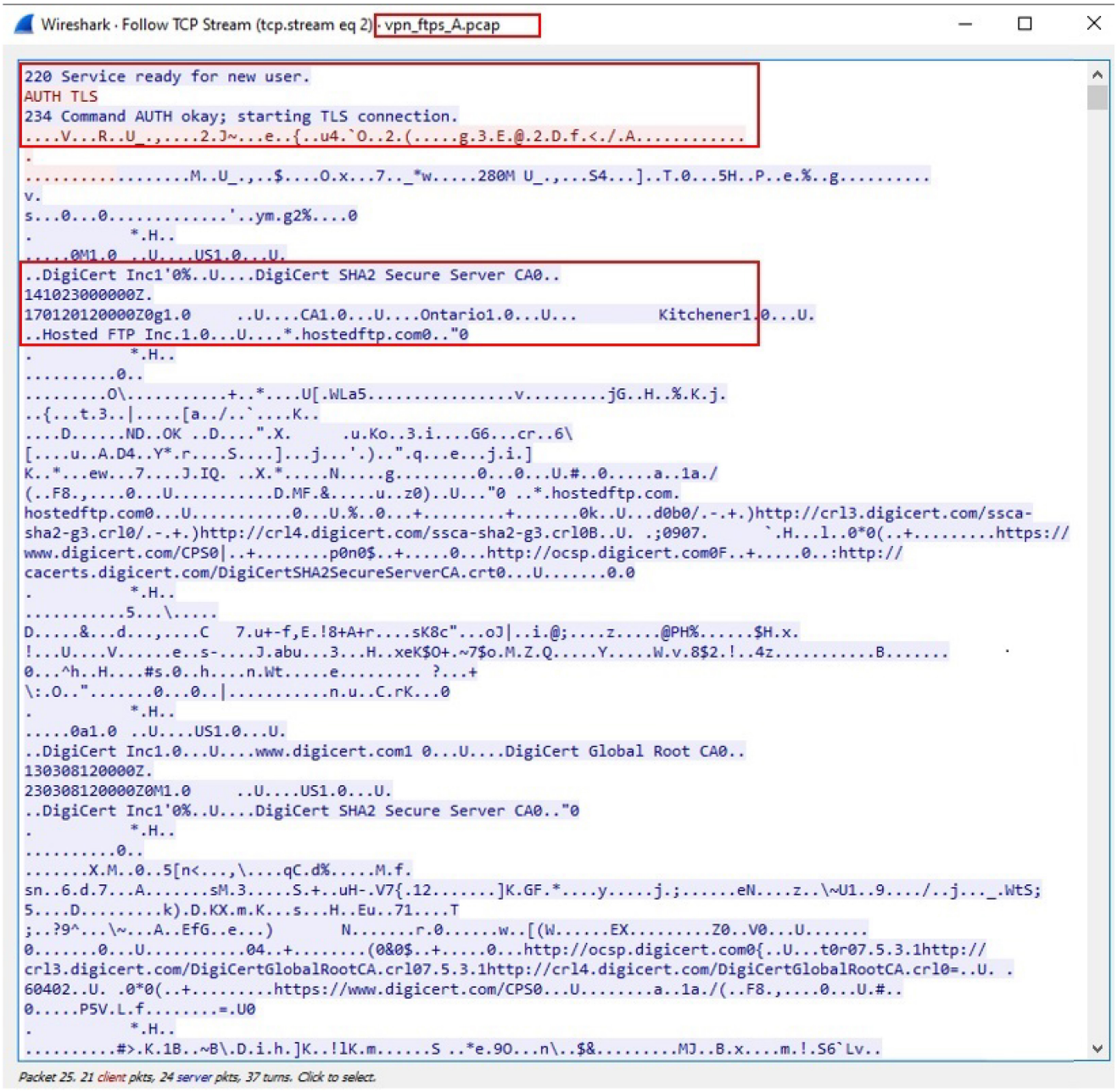


Fig. 10. Syntax of the FTP protocol is visible in the payload of data.

and 83.60% (compared to 93.40%) accuracy in application classification (experiment six). This shows that application-level accuracy drops significantly after removing the last four features, which shows the major contribution of those features.

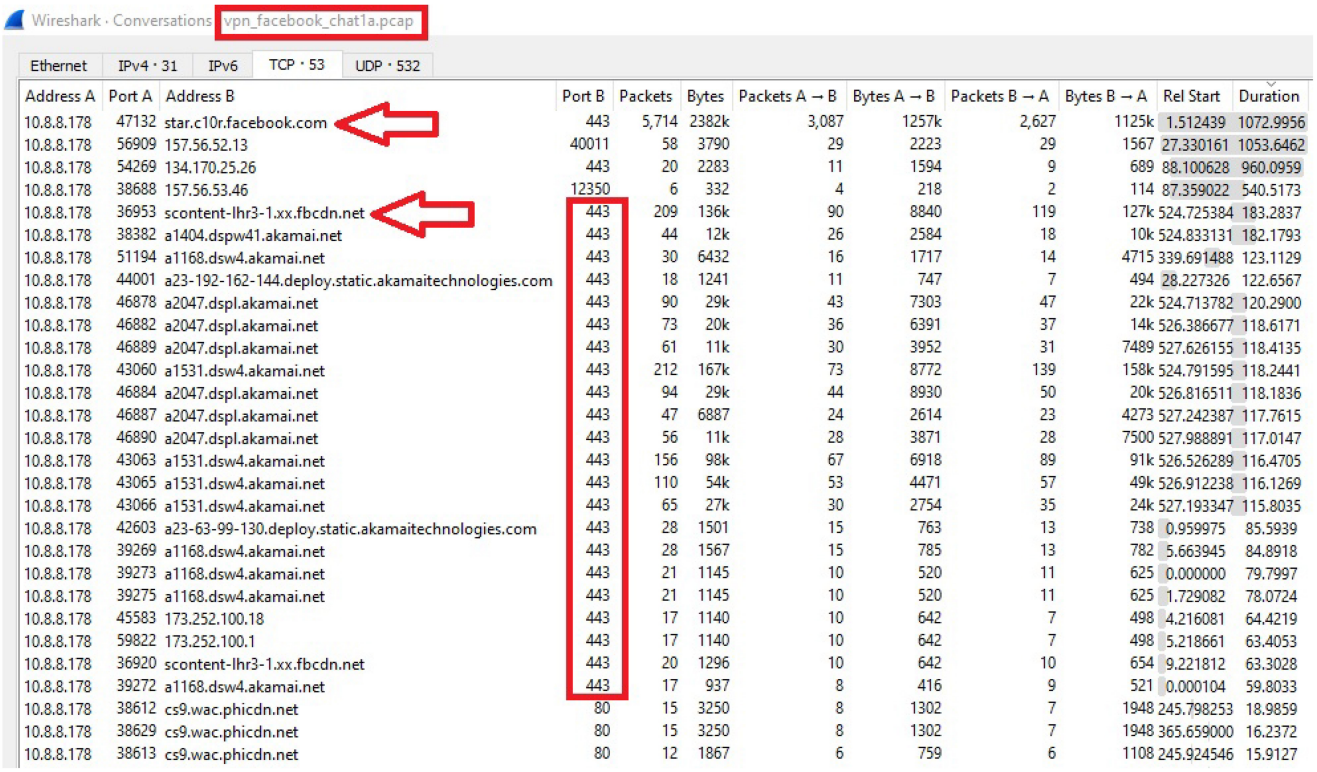
Note that to obtain a granular view of the performance, we examined each application's test data against the model; the results can be found in Table X.

We also represent the top 10 features for each experiment in the Appendix.

VI. Discussion

Prior research [16], [29], [31] only demonstrated the proposed method's effectiveness on the existing dataset, which has limitations. Here we produced a new balanced dataset with data from a larger number of applications and from different geographical locations, which is better suited to today's applications, and used it to demonstrate the effectiveness of our proposed approach. To demonstrate the generic and universal nature of our approach, we also assessed our approach's ability to identify applications in non-VPN-encrypted network traffic (experiments 5 & 6) as well as IoT devices; the results of this experiment (experiment 7) are presented at the end of this section).

We used a sliding window time-series approach which enables our method to identity any type of traffic with less data than that needed by other methods, i.e., the number of seconds of traffic needed is equal to the length of the sliding window. In other approaches one might need to wait for the flow's



Address A	Port A	Address B	Port B	Packets	Bytes	Packets A → B	Bytes A → B	Packets B → A	Bytes B → A	Rel Start	Duration
10.8.8.178	47132	star.c10r.facebook.com	443	5,714	2382k	3,087	1257k	2,627	1125k	1.512439	1072.9956
10.8.8.178	56909	157.56.52.13	40011	58	3790	29	2223	29	1567	27.330161	1053.6462
10.8.8.178	54269	134.170.25.26	443	20	2283	11	1594	9	689	88.100628	960.0959
10.8.8.178	38688	157.56.53.46	12350	6	332	4	218	2	114	87.359022	540.5173
10.8.8.178	36953	scontent-lhr3-1.xx.fbcdn.net	443	209	136k	90	8840	119	127k	524.725384	183.2837
10.8.8.178	38382	a1404.dspw41.akamai.net	443	44	12k	26	2584	18	10k	524.833131	182.1793
10.8.8.178	51194	a1168.dsw4.akamai.net	443	30	6432	16	1717	14	4715	339.691488	123.1129
10.8.8.178	44001	a23-192-162-144.deploy.static.akamaitechnologies.com	443	18	1241	11	747	7	494	28.227326	122.6567
10.8.8.178	46878	a2047.dspl.akamai.net	443	90	29k	43	7303	47	22k	524.713782	120.2900
10.8.8.178	46882	a2047.dspl.akamai.net	443	73	20k	36	6391	37	14k	526.386677	118.6171
10.8.8.178	46889	a2047.dspl.akamai.net	443	61	11k	30	3952	31	7489	527.626155	118.4135
10.8.8.178	43060	a1531.dsw4.akamai.net	443	212	167k	73	8772	139	158k	524.791595	118.2441
10.8.8.178	46884	a2047.dspl.akamai.net	443	94	29k	44	8930	50	20k	526.816511	118.1836
10.8.8.178	46887	a2047.dspl.akamai.net	443	47	6887	24	2614	23	4273	527.242387	117.7615
10.8.8.178	46890	a2047.dspl.akamai.net	443	56	11k	28	3871	28	7500	527.988891	117.0147
10.8.8.178	43063	a1531.dsw4.akamai.net	443	156	98k	67	6918	89	91k	526.526289	116.4705
10.8.8.178	43065	a1531.dsw4.akamai.net	443	110	54k	53	4471	57	49k	526.912238	116.1269
10.8.8.178	43066	a1531.dsw4.akamai.net	443	65	27k	30	2754	35	24k	527.193347	115.8035
10.8.8.178	42603	a23-63-99-130.deploy.static.akamaitechnologies.com	443	28	1501	15	763	13	738	0.959975	85.5939
10.8.8.178	39269	a1168.dsw4.akamai.net	443	28	1567	15	785	13	782	5.663945	84.8918
10.8.8.178	39273	a1168.dsw4.akamai.net	443	21	1145	10	520	11	625	0.000000	79.7997
10.8.8.178	39275	a1168.dsw4.akamai.net	443	21	1145	10	520	11	625	1.729082	78.0724
10.8.8.178	45583	173.252.100.18	443	17	1140	10	642	7	498	4.216081	64.4219
10.8.8.178	59822	173.252.100.1	443	17	1140	10	642	7	498	5.218661	63.4053
10.8.8.178	36920	scontent-lhr3-1.xx.fbcdn.net	443	20	1296	10	642	10	654	9.221812	63.3028
10.8.8.178	39272	a1168.dsw4.akamai.net	443	17	937	8	416	9	521	0.000104	59.8033
10.8.8.178	38612	cs9.wac.phicdn.net	80	15	3250	8	1302	7	1948	245.798253	18.9859
10.8.8.178	38629	cs9.wac.phicdn.net	80	15	3250	8	1302	7	1948	365.659000	16.2372
10.8.8.178	38613	cs9.wac.phicdn.net	80	12	1867	6	759	6	1108	245.924546	15.9127

Fig. 11. PCAP shows multiple flows with well-known domain names and ports.

TABLE XIII
MOST IMPORTANT FEATURES IN EXPERIMENT 1

Series	Sub-Series	Feature	Difference (%)
packet sizes	outgoing packets	fw_1250_pkts	18.35
packet sizes	2D statistics based on incoming and outgoing sub-series	Magnitude	7.69
packet sizes	incoming packets	bw_500_pkts	7.38
packet sizes	incoming packets	Standard deviation	2.68
packet sizes	total packets	Mean	2.44
packet sizes	2D statistics based on incoming and outgoing sub-series	Covariance	1.44
packet sizes	outgoing packets	fw_1500_pkts	1.44
time delta	total packets	Standard deviation	1.30
packet sizes	outgoing packets	Mean	1.23
packet sizes	outgoing packets	Standard deviation	1.22

TABLE XIV
MOST IMPORTANT FEATURES IN EXPERIMENT 2

Series	Sub-Series	Feature	Difference (%)
packet sizes	2D statistics based on incoming and outgoing sub-series	Radius	15.98
packet sizes	incoming packets	Standard deviation	12.32
packet sizes	outgoing packets	fw_1250_pkts	10.57
packet sizes	incoming packets	Mean	9.78
packet sizes	2D statistics based on incoming and outgoing sub-series	Magnitude	8.40
packet sizes	outgoing packets	Standard deviation	7.13
packet sizes	outgoing packets	Mean	6.27
packet sizes	total packets	Mean	5.75
accumulated directional packet sizes	outgoing packets	Mean	5.58
packet sizes	incoming packets	bw_500_pkts	5.06

completion or for a certain number of packets to be exchanged, which can often be time consuming. Our aim in proposing this approach was to provide a time-series-based approach which is generic in nature and can be applied to different network scenarios, irrespective of the domain (e.g., IoT and computer network), protocol (e.g., TCP and UDP), and the type of encryption (e.g., VPN-encrypted, TLS-encrypted, and non-encrypted). As our approach differs from existing approaches in that it is based on time series, we could not directly compare the methods' accuracy in our performance evaluation; in existing flow-based approaches if there are 10 flows in the traffic there will be 10 data instances, whereas in our approach, the number of data instances can vary based on the time; therefore, we were unable to compare the performance of our approach directly with the performance of existing methods. However, the accuracy reported in this paper is comparable or better than the accuracy obtained by other methods in most of the examined scenarios.

TABLE XV
MOST IMPORTANT FEATURES IN EXPERIMENT 3 (JAPAN)

Series	Sub-Series	Feature	Difference (%)
packet sizes	2D statistics based on incoming and outgoing sub-series	Magnitude	6.67
packet sizes	2D statistics based on incoming and outgoing sub-series	Radius	3.09
packet sizes	outgoing packets	fw_1000_pkts	2.77
time delta	2D statistics based on incoming and outgoing sub-series	Covariance	0.98
packet sizes	2D statistics based on incoming and outgoing sub-series	Correlation coefficient	0.81
packet sizes	incoming packets	bw_1250_pkts	0.49
accumulated directional packet sizes	outgoing packets	Mean	0.45
time delta	total packets	Mean	0.42
time delta	2D statistics based on incoming and outgoing sub-series	Correlation coefficient	0.34
packet sizes	outgoing packets	fw_250_pkts	0.34

TABLE XVI
MOST IMPORTANT FEATURES IN EXPERIMENT 3 (ENGLAND)

Series	Sub-Series	Feature	Difference (%)
packet sizes	incoming packets	bw_500_pkts	22.66
packet sizes	incoming packets	Mean	6.44
packet sizes	incoming packets	Standard deviation	4.63
packet sizes	total packets	Standard deviation	1.57
packet sizes	outgoing packets	Mean	1.23
accumulated directional packet sizes	total packets	Standard deviation	1.16
packet sizes	outgoing packets	fw_500_pkts	1.12
packet sizes	outgoing packets	fw_1500_pkts	1.11
accumulated directional packet sizes	total packets	Mean	1.05
packet sizes	2D statistics based on incoming and outgoing sub-series	Covariance	0.95

TABLE XVII
MOST IMPORTANT FEATURES IN EXPERIMENT 3 (CANADA)

Series	Sub-Series	Feature	Difference (%)
packet sizes	outgoing packets	fw_1250_pkts	17.06
packet sizes	incoming packets	Mean	1.42
time delta	total packets	Standard deviation	1.11
packet sizes	outgoing packets	fw_250_pkts	0.85
packet sizes	outgoing packets	fw_750_pkts	0.85
packet sizes	outgoing packets	Mean	0.49
packet sizes	incoming packets	bw_1250_pkts	0.43
packet sizes	incoming packets	bw_1500_pkts	0.27
packet sizes	total packets	Mean	0.24
packet sizes	incoming packets	bw_500_pkts	0.23

TABLE XVIII
MOST IMPORTANT FEATURES IN EXPERIMENT 4 (JAPAN)

Series	Sub-Series	Feature	Difference (%)
packet sizes	outgoing packets	Standard deviation	9.60
packet sizes	2D statistics based on incoming and outgoing sub-series	Radius	9.12
packet sizes	outgoing packets	Mean	6.36
packet sizes	total packets	Mean	5.48
packet sizes	incoming packets	Standard deviation	4.42
packet sizes	incoming packets	bw_1250_pkts	4.20
packet sizes	2D statistics based on incoming and outgoing sub-series	Magnitude	3.14
packet sizes	2D statistics based on incoming and outgoing sub-series	Correlation coefficient	2.15
packet sizes	incoming packets	Mean	1.79
accumulated directional packet sizes	outgoing packets	Mean	1.49

Experiment 7: Classification of IoT Device Type

In this experiment, we used a subset of the IoTTrace [17] dataset (specifically, traffic from October 23 and 24). We took into account IoT devices whose network traffic lasted for more than 3000 seconds (i.e., 50 minutes) to train two models (each model was trained for a day). We used 8000 seconds of traffic from each IoT device in the experiment. We were unable to use the entire dataset, because the data was not continuous, as is needed for training in our approach. However, our aim was only to perform a proof of concept demonstrating the applicability of our approach on a different type of dataset. The results of the two models trained and tested on traffic from October 23 and 24 are presented in Tables XI and XII respectively. In this experiment we removed the features (for example, *fw_1250_packets*) that had a standard deviation value of zero, since they had no impact on the performance and created an error (division by zero) when performing standardization. Also, as mentioned earlier we included the last four features mentioned in Table III in this experiment. We note that although we used two different *Netatmo* devices, the model was able to bifurcate their traffic correctly despite both devices coming from the same manufacturer.

To see the impact of the last four features (i.e., feature numbers 44 to 47), we reran this experiment for a sliding window of 60 seconds without those four features and obtained 97.80% (compared to 97.80%) and 98.90% (compared to 99.80%) accuracy for the October 23 and October 24 datasets respectively. This shows that removing the last four features had no, or very minimal, impact on performance in this case.

TABLE XIX
MOST IMPORTANT FEATURES IN EXPERIMENT 4 (ENGLAND)

Series	Sub-Series	Feature	Difference (%)
packet sizes	incoming packets	bw_500_pkts	19.05
packet sizes	incoming packets	Mean	11.75
packet sizes	incoming packets	Standard deviation	6.57
packet sizes	outgoing packets	fw_500_pkts	6.44
packet sizes	outgoing packets	Mean	6.05
accumulated directional packet sizes	outgoing packets	Mean	4.48
packet sizes	total packets	Standard deviation	3.13
packet sizes	outgoing packets	fw_250_pkts	3.00
packet sizes	incoming packets	bw_750_pkts	2.97
packet sizes	2D statistics based on incoming and outgoing sub-series	Radius	2.88

TABLE XX
MOST IMPORTANT FEATURES IN EXPERIMENT 4 (CANADA)

Series	Sub-Series	Feature	Difference (%)
packet sizes	incoming packets	Standard deviation	9.50
packet sizes	outgoing packets	fw_1250_pkts	9.42
packet sizes	outgoing packets	Mean	7.64
packet sizes	total packets	Mean	6.27
packet sizes	incoming packets	Mean	5.32
time delta	total packets	Standard deviation	2.49
packet sizes	outgoing packets	Standard deviation	2.31
packet sizes	outgoing packets	fw_750_pkts	1.56
time delta	total packets	Mean	1.44

TABLE XXI
MOST IMPORTANT FEATURES IN EXPERIMENT 5

Series	Sub-Series	Feature	Difference (%)
additional features	-	uniq_dst_ips	19.82
additional features	-	uniq_src_ips	13.25
packet sizes	incoming packets	bw_250_pkts	11.95
packet sizes	total packets	Mean	9.77
additional features	-	unique_src_ports	8.47
additional features	-	unique_dst_ports	7.92
packet sizes	outgoing packets	Mean	7.77
time delta	outgoing packets	Standard deviation	4.26
packet sizes	total packets	Standard deviation	3.47
packet sizes	incoming packets	bw_1000_pkts	3.41

TABLE XXII
MOST IMPORTANT FEATURES IN EXPERIMENT 6

Series	Sub-Series	Feature	Difference (%)
additional features	-	uniq_dst_ips	34.76
additional features	-	uniq_src_ips	23.72
additional features	-	unique_dst_ports	19.74
packet sizes	incoming packets	bw_1000_pkts	17.17
additional features	-	unique_src_ports	15.91
packet sizes	incoming packets	bw_250_pkts	11.12
packet sizes	outgoing packets	Mean	9.65
accumulated directional packet sizes	2D statistics based on incoming and outgoing sub-series	Correlation coefficient	6.30
packet sizes	total packets	Mean	6.12
packet sizes	incoming packets	bw_500_pkts	5.28

TABLE XXIII
MOST IMPORTANT FEATURES IN EXPERIMENT 7 (OCTOBER 23)

Series	Sub-Series	Feature	Difference (%)
additional features	-	uniq_src_ips	37.06
additional features	-	uniq_dst_ips	33.93
packet sizes	total packets	Mean	23.27
additional features	-	unique_dst_ports	11.32
packet sizes	outgoing packets	fw_500_pkts	11.28
additional features	-	unique_src_ports	10.61
packet sizes	total packets	Standard deviation	8.39
packet sizes	outgoing packets	Standard deviation	8.05
packet sizes	incoming packets	bw_250_pkts	7.94
packet sizes	outgoing packets	Mean	7.54

TABLE XXIV
MOST IMPORTANT FEATURES IN EXPERIMENT 7 (OCTOBER 24)

Series	Sub-Series	Feature	Difference (%)
additional features	-	uniq_src_ips	9.37
additional features	-	uniq_dst_ips	7.25
packet sizes	outgoing packets	fw_250_pkts	7.17
packet sizes	incoming packets	bw_250_pkts	5.30
additional features	-	unique_dst_ports	4.29
additional features	-	unique_src_ports	4.18
packet sizes	incoming packets	bw_500_pkts	2.47
packet sizes	outgoing packets	fw_500_pkts	1.10
packet sizes	outgoing packets	Standard deviation	0.83
packet sizes	outgoing packets	Mean	0.66

VII. CONCLUSION

In this paper, we proposed a time-series-based approach for the classification of VPN-encrypted network traffic at the application- and category-levels. To evaluate our approach we created a new dataset that consists of the traffic of different applications captured from three different geographical

Algorithm 1 Feature Extraction

Input:
- **TSV_file:** TSV file containing parsed packet data
- **window_length:** List of time window lengths in seconds (e.g. 30, 40, 50, 60 etc.) for feature extraction

Output:
- *features_df:* Dataframe containing extracted features

Algorithm:

Load parsed packet data from TSV file into dataframe
Determine packet direction based on source port
Initialize an empty list for statistics
for each time sample in window_length **do**
  Extract data sample within the specified time window from dataframe
  Compute time delta features:
  - Bidirectional delta ratios between consecutive packets: $(r^{bi}_{\Delta t})_i = \frac{\Delta t_{\text{in},i}}{\Delta t_{\text{out},i+1}}$
  - Ingoing delta ratios between consecutive packets: $(r^{in}_{\Delta t})_i = \frac{\Delta t_{\text{in},i}}{\Delta t_{\text{in},i+1}}$
  - Outgoing delta ratios between consecutive packets: $(r^{out}_{\Delta t})_i = \frac{\Delta t_{\text{out},i}}{\Delta t_{\text{out},i+1}}$

  Compute packet size features:
  - Bidirectional size ratios between consecutive packets: $(r^{bi}_{size})_i = \frac{\text{size}_{\text{in},i}}{\text{size}_{\text{out},i+1}}$
  - Ingoing size ratios between consecutive packets: $(r^{in}_{size})_i = \frac{\text{size}_{\text{in},i}}{\text{size}_{\text{in},i+1}}$
  - Outgoing size ratios between consecutive packets: $(r^{out}_{size})_i = \frac{\text{size}_{\text{out},i}}{\text{size}_{\text{out},i+1}}$

  Compute accumulated packet size features:
  - Accumulated bidirectional size ratios: $(r^{bi}_{\sum size})_i = \frac{\sum_{j=1}^{i} \text{size}_{\text{in},j}}{\sum_{j=1}^{i+1} \text{size}_{\text{out},j}}$
  - Accumulated ingoing size ratios: $(r^{in}_{\sum size})_i = \frac{\sum_{j=1}^{i} \text{size}_{\text{in},j}}{\sum_{j=1}^{i+1} \text{size}_{\text{in},j}}$
  - Accumulated outgoing size ratios: $(r^{out}_{\sum size})_i = \frac{\sum_{j=1}^{i} \text{size}_{\text{out},j}}{\sum_{j=1}^{i+1} \text{size}_{\text{out},j}}$

  Compute additional features
  Append all computed features to the statistics list
end for
Convert the statistics list into a new dataframe (features_df)
Return features_df

locations. Such a dataset is needed by the research community, since we found that the existing dataset has several limitations. We also demonstrated the generic nature of the proposed approach by evaluating it in several other scenarios, such as classification of non-VPN-encrypted traffic and IoT device type. It should also be noted that along with organizations and ISPs, methods of this type can also be used by adversaries for reconnaissance of their targets. In future research, we plan to apply our approach in scenarios where there is network traffic flowing simultaneously from different applications within the same VPN tunnel. We also plan to train our approach on data from a particular country and test it against data from different countries.

APPENDIX

The tables below present the 10 most important features for each experiment. The "Difference (%)" column in each table represents the difference between the baseline validation accuracy (with all features) and the new validation accuracy (without the feature mentioned).

REFERENCES

[1] Statista. "Topic: Internet usage worldwide." [Online]. Available: https://www.statista.com/topics/1145/internet-usage-worldwide/#topicHeader__wrapper

[2] R. Subramanian, "The growth of global Internet censorship and circumvention: A survey," *Commun. Int. Inf. Manag. Assoc.*, vol. 11, no. 2, p. 6, 2011.

[3] F. Li, A. A. Niaki, D. Choffnes, P. Gill, and A. Mislove, "A large-scale analysis of deployed traffic differentiation practices," in *Proc. ACM Special Interest Group Data Commun.*, 2019, pp. 130–144.

[4] N. Weaver, C. Kreibich, and V. Paxson, "Redirecting *DNS* for Ads and profit," in *Proc. USENIX Workshop Free Open Commun. Internet (FOCI)*, 2011, p. 12.

[5] A. M. Kakhki et al., "Identifying traffic differentiation in mobile networks," in *Proc. Internet Meas. Conf.*, 2015, pp. 239–251.

[6] T. Garrett, L. E. Setenareski, L. M. Peres, L. C. Bona, and E. P. Duarte, "Monitoring network neutrality: A survey on traffic differentiation detection," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 3, pp. 2486–2517, 3rd Quart., 2018.

[7] R. S. Raman, L. Evdokimov, E. Wurstrow, J. A. Halderman, and R. Ensafi, "Investigating large scale HTTPS interception in Kazakhstan," in *Proc. ACM Internet Meas. Conf.*, 2020, pp. 125–132.

[8] M. T. Khan, J. DeBlasio, G. M. Voelker, A. C. Snoeren, C. Kanich, and N. Vallina-Rodriguez, "An empirical analysis of the commercial VPN ecosystem," in *Proc. Internet Meas. Conf.*, 2018, pp. 443–456.

[9] M. Ikram, N. Vallina-Rodriguez, S. Seneviratne, M. A. Kaafar, and V. Paxson, "An analysis of the privacy and security risks of android VPN permission-enabled APPs," in *Proc. Internet Meas. Conf.*, 2016, pp. 349–364.

[10] T. Sharma and M. Bashir, "Privacy apps for smartphones: An assessment of users' preferences and limitations," in *Proc. Int. Conf. Human–Comput. Interact.*, 2020, pp. 533–546.

[11] J. Zhang, X. Chen, Y. Xiang, W. Zhou, and J. Wu, "Robust network traffic classification," *IEEE/ACM Trans. Netw.*, vol. 23, no. 4, pp. 1257–1270, Aug. 2015.

[12] T. T. Nguyen and G. Armitage, "A survey of techniques for Internet traffic classification using machine learning," *IEEE Commun. Surveys Tuts.*, vol. 10, no. 4, pp. 56–76, 4th Quart., 2008.

[13] A. Callado et al., "A survey on Internet traffic identification," *IEEE Commun. Surveys Tuts.*, vol. 11, no. 3, pp. 37–52, 3rd Quart., 2009.

[14] N. Al Khater and R. E. Overill, "Network traffic classification techniques and challenges," in *Proc. IEEE 10th Int. Conf. Digit. Inf. Manag. (ICDIM)*, 2015, pp. 43–48.

[15] H. I. Fawaz et al., "InceptionTime: Finding AlexNet for time series classification," *Data Min. Knowl. Disc.*, vol. 34, no. 6, pp. 1936–1962, 2020.

[16] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related," in *Proc. 2nd Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, 2016, pp. 407–414.

[17] A. Sivanathan et al., "Classifying IoT devices in smart environments using network traffic characteristics," *IEEE Trans. Mobile Comput.*, vol. 18, no. 8, pp. 1745–1759, Aug. 2018.

[18] "Service name and transport protocol port number registry." 2019. [Online]. Available: https://www.iana.org/assignments/service-names-port-numbers/service-names-port-numbers.xhtml?&page=1

[19] P. Schneider, *TCP/IP Traffic Classification Based on Port Numbers*, Harvard Univ., Cambridge, MA, USA, 1997.

[20] G. Cheng and S. Wang, "Traffic classification based on port connection pattern," in *Proc. Int. Conf. Comput. Sci. Service Syst. (CSSS)*, 2011, pp. 914–917.

[21] Q. Zhang, Y. Ma, J. Wang, and X. Li, "UDP traffic classification using most distinguished port," in *Proc. 16th Asia–Pac. Netw. Oper. Manag. Symp.*, 2014, pp. 1–4.

[22] M. Finsterbusch, C. Richter, E. Rocha, J.-A. Muller, and K. Hanssgen, "A survey of payload-based traffic classification approaches," *IEEE Commun. Surveys Tuts.*, vol. 16, no. 2, pp. 1135–1156, 2nd Quart., 2013.

[23] H.-K. Lim, J.-B. Kim, K. Kim, Y.-G. Hong, and Y.-H. Han, "Payload-based traffic classification using multi-layer LSTM in software defined networks," *Appl. Sci.*, vol. 9, no. 12, p. 2550, 2019. [Online]. Available: https://www.mdpi.com/2076-3417/9/12/2550

[24] F. Risso, M. Baldi, O. Morandi, A. Baldini, and P. Monclus, "Lightweight, payload-based traffic classification: An experimental evaluation," in *Proc. IEEE Int. Conf. Commun.*, 2008, pp. 5869–5875.

[25] Z. Fan and R. Liu, "Investigation of machine learning based network traffic classification," in *Proc. Int. Symp. Wireless Commun. Syst. (ISWCS)*, 2017, pp. 1–6.

[26] S. Özdel, Ç. Ateş, P. D. Ateş, M. Koca, and E. Anarım, "Payload-based network traffic analysis for application classification and intrusion detection," in *Proc. 30th Eur. Signal Process. Conf. (EUSIPCO)*, 2022, pp. 638–642.

[27] J. Kotak and Y. Elovici, "IoT device identification using deep learning," in *Proc. Comput. Intell. Security Inf. Syst. Conf.*, 2019, pp. 76–86.

[28] M. Lotfollahi, M. J. Siavoshani, R. S. H. Zade, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *Soft Comput.*, vol. 24, no. 3, pp. 1999–2012, 2020.

[29] L. Vu et al., "Time series analysis for encrypted traffic classification: A deep learning approach," in *Proc. IEEE 18th Int. Symp. Commun. Inf. Technol. (ISCIT)*, 2018, pp. 121–126.

[30] S. Soleymanpour, H. Sadr, and M. N. Soleimandarabi, "CSCNN: Cost-sensitive convolutional neural network for encrypted traffic classification," *Neural Process. Lett.*, vol. 53, no. 5, pp. 3497–3523, 2021.

[31] G. Aceto, D. Ciuonzo, A. Montieri, and A. Pescapé, "DISTILLER: Encrypted traffic classification via multimodal multitask deep learning," *J. Netw. Comput. Appl.*, vols. 183–184, Jun. 2021, Art. no. 102985.

[32] M. Al-Fayoumi, M. Al-Fawa'reh, and S. Nashwan, "VPN and non-VPN network traffic classification using time-related features," *Comput. Materials Continua*, vol. 72, no. 2, p. 12, 2022.

[33] S. Roy, T. Shapira, and Y. Shavitt, "Fast and lean encrypted Internet traffic classification," *Comput. Commun.*, vol. 186, pp. 166–173, Mar. 2022.

[34] Z. Shi, N. Luktarhan, Y. Song, and G. Tian, "BFCN: A novel classification method of encrypted traffic based on BERT and CNN," *Electronics*, vol. 12, no. 3, p. 516, 2023.

[35] S. Ramraj and G. Usha, "Hybrid feature learning framework for the classification of encrypted network traffic," *Connection Sci.*, vol. 35, no. 1, 2023, Art. no. 2197172.

[36] S. Lv, C. Wang, Z. Wang, S. Wang, B. Wang, and Y. Zhang, "AAE-DSVDD: A one-class classification model for VPN traffic identification," *Comput. Netw.*, vol. 236, Nov. 2023, Art. no. 109990.

[37] G. Abbas, U. Farooq, P. Singh, S. S. Khurana, and P. Singh, "Feature engineering and ensemble learning-based classification of VPN and non-VPN-based network traffic over temporal features," *SN Comput. Sci.*, vol. 4, no. 5, p. 546, 2023.

[38] Z. Wang et al. "An effective real-time traffic classification method using convolutional neural network." 2023. [Online]. Available: https://ouci.dntb.gov.ua/en/works/4ggEk2B4/

[39] Y. Xu, J. Cao, K. Song, Q. Xiang, and G. Cheng, "FastTraffic: A lightweight method for encrypted traffic fast classification," *Comput. Netw.*, vol. 235, Nov. 2023, Art. no. 109965.

[40] R. T. Elmaghraby, N. M. A. Aziem, M. A. Sobh, and A. M. Bahaa-Eldin, "Encrypted network traffic classification based on machine learning," *Ain Shams Eng. J.*, vol. 15, no. 2, 2024, Art. no. 102361.

[41] S. Soleymanpour, H. Sadr, and H. Beheshti, "An efficient deep learning method for encrypted traffic classification on the Web," in *Proc. IEEE 6th Int. Conf. Web Res. (ICWR)*, 2020, pp. 209–216.

[42] A. Iliyasu and H. Deng, "Semi-supervised encrypted traffic classification with deep convolutional generative adversarial networks," *IEEE Access*, vol. 8, pp. 118–126, 2019.

[43] R. Nigmatullin, A. Ivchenko, and S. Dorokhin, "Differentiation of sliding rescaled ranges: New approach to encrypted and VPN traffic detection," in *Proc. Int. Conf. Eng. Telecommun. (En&T)*, 2020, pp. 1–5.

[44] B. Appiah, A. Sackey, O.-A. Kwabena, J. B. Ansuura, and P. Buah, "Fusion dilated CNN for encrypted Web traffic classification," *Int. J. Netw. Security*, vol. 24, pp. 733–740, Jul. 2022.

[45] D. Wei, F. Shi, and S. Dhelim, "A self-supervised learning model for unknown internet traffic identification based on surge period," *Future Internet*, vol. 14, p. 289, Oct. 2022.

[46] M. Pathmaperuma, Y. Rahulamathavn, S. Dogan, and A. Kondoz, "Deep learning for encrypted traffic classification and unknown data detection," *Sensors*, vol. 22, p. 7643, Oct. 2022.

[47] Y. Li, F. Wang, and S. Chen, "VPN traffic identification based on tunneling protocol characteristics," in *Proc. IEEE CCET*, Aug. 2022, pp. 150–156.

[48] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "KitSune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Netw. Distrib. Syst. Security Symp.*, 2018, pp. 1–8.

[49] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. A. Alemi, "Inception-v4, inception-ResNet and the impact of residual connections on learning," in *Proc. 31st AAAI Conf. Artif. Intell.*, 2017, pp. 4278–4284.

[50] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.